

UNIVERSITY OF OKLAHOMA

GRADUATE COLLEGE

ADAPTING SPARSE COLLECTIVE COMMUNICATION
ALGORITHMS FOR STRUCTURED GRAPHS AND MATRICES

A THESIS

SUBMITTED TO THE GRADUATE FACULTY

in partial fulfillment of the requirements for the

Degree of

MASTER OF SCIENCE

By DYLAN ZEMLIN

Norman, Oklahoma

2026

**ADAPTING SPARSE COLLECTIVE COMMUNICATION
ALGORITHMS FOR STRUCTURED GRAPHS AND MATRICES**

A THESIS APPROVED FOR THE
SCHOOL OF COMPUTER SCIENCE

BY THE COMMITTEE CONSISTING OF

Dr. Richard Veras, Chair

Dr. Sridhar Radhakrishnan

Dr. Le Gruenwald

© Copyright by DYLAN ZEMLIN 2026

All Rights Reserved.

Acknowledgments

The computing for this project was performed at the OU Supercomputing Center for Education & Research (OSCER) at the University of Oklahoma (OU).

This material is based upon work supported by the National Science Foundation under Grant No. 2440646.

Table of Contents

Acknowledgments	iv
List of Figures	xi
List of Tables	xv
Abstract	xvi
1 Introduction	1
1.1 Structure and Patterns	3
1.2 Distributed Sparse Matrices	4
1.3 Thesis Outline	5
2 Background	7
2.1 Sparse Matrix Representations	7
2.1.1 Coordinate (COO) Format	7
2.1.2 Compressed Sparse Row	8
2.1.3 Compressed Sparse Column	10
2.2 Sparsity Patterns and Matrix Structure	11
2.2.1 Uniform Random Sparsity	11
2.2.2 Banded Sparsity	13
2.2.3 Block Sparsity	13
2.2.4 Kronecker Product Sparsity	15
2.3 Distributed Memory Parallelism (MPI)	17

2.4	Collective Communications	18
2.4.1	MST Broadcast, Scatter, Gather, and Reduce	19
2.4.1.1	Broadcast	20
2.4.1.2	Scatter	20
2.4.1.3	Gather	21
2.4.1.4	Reduce	22
2.4.2	Bucket All Gather and Reduce Scatter	23
2.5	Distributed Matrix Multiplication (SUMMA)	26
2.5.1	Block Size Tradeoffs	27
2.5.2	Communication Cost	28
3	Sparse Collective Communications	30
3.1	Adapting Collectives or Sparse Data	30
3.1.1	Variable Message Sizes	31
3.1.2	Metadata Overhead	31
3.2	Data Dependent Patterns	32
3.3	Communication Cost Analysis	33
3.3.1	Cost Model for Sparse Data	33
3.3.2	Sparsity Level vs Volume	34
3.3.3	Load Imbalance across Processes	35
3.4	Expected Effects by Sparsity Pattern	37
3.5	Scaling Behavior	38
3.5.1	Strong Scaling	38
3.5.2	Weak Scaling	38
3.6	Summary	39
4	Sparse SUMMA	40

4.1	From Dense to Sparse SUMMA	40
4.1.1	Overview	40
4.1.2	Sparsity Challenges	41
4.2	Storage Format Implications	42
4.2.1	CSR and A-Panel Broadcasts	42
4.2.2	CSC and B-Panel Broadcasts	42
4.2.3	Format Tradeoffs	43
4.3	Matrix Shape and Process Grids	43
4.3.1	Square vs Rectangular Matrices	43
4.4	Sparsity Pattern Effects	44
4.4.1	Communication Bottlenecks vs Local Multiplication	44
4.4.2	Expected Behaviors by Pattern	45
4.5	Summary	46
5	Methodology and Experimental Setup	47
5.1	Matrix Shape and Sparsity	47
5.2	Parallelism	48
5.3	Hardware and Software	49
5.4	Metrics and Instrumentation	49
5.4.1	Custom Profiling Tool	49
5.4.1.1	PMPI	52
5.4.1.2	Output	52
6	Results and Analysis	55
6.1	Experiment Design and Variability	55
6.2	Baselines	56
6.3	Collective Communication Results	56

6.3.1	Impact of Sparsity Level	56
6.3.1.1	MST Collectives	56
6.3.1.2	Bucket Collectives	61
6.3.2	Impact of Sparsity Pattern	64
6.3.2.1	MST Collectives	65
6.3.2.2	Bucket Collectives	67
6.3.3	Scaling Behavior	68
6.3.4	Combined Effects	69
6.4	SUMMA Results	69
6.4.1	Impact of Sparsity Level	69
6.4.2	Impact of Sparsity Pattern	71
6.4.3	Impact of Matrix Dimensions	72
6.4.4	Impact of Storage Format	73
6.4.5	Communication vs Computation Bottlenecks	74
6.4.6	Load Imbalance Observations	75
6.4.7	Combined Effects	76
6.5	Discussion	77
6.5.1	Storage Format Selection	77
6.5.2	Sparsity Pattern and Algorithm Selection	78
6.5.3	Communication as the Dominant Bottleneck	78
6.5.4	Scaling Behavior	78
6.5.5	Summary	78
7	Distributed Sparse Loading and Storage	79
7.1	The Distributed Loading Problem	79
7.1.1	Loading Strategies	80

7.1.2	Single Process Read and Broadcast/Scatter	82
7.1.2.1	Row Based Partitioning	82
7.1.2.2	Nonzero Balanced Partitioning	82
7.1.2.3	Tradeoffs	83
7.1.3	Storage Format Conversions	83
7.1.3.1	HDF5	83
7.1.3.2	Zarr	83
7.2	Summary	84
8	Visualization and STEM Communication	85
8.1	Communicating Sparse Structure	85
8.2	Communicating Sparse Algorithms	86
8.3	Design Principles for Technical Figures	86
8.4	Figures in this Thesis	86
8.4.1	Tool Implementation	86
8.4.2	Matrix Structure Diagrams	87
8.4.3	Collective Communication Diagrams	87
8.4.4	Performance and Scaling Charts	89
8.4.5	Summary	89
9	Discussion and Future Work	90
9.1	Summary of Contributions	90
9.2	Pattern-Algorithm Summary	91
9.3	Limitations	91
9.4	Future Work	91
9.4.1	Extending to Other Sparse Formats	91
9.4.2	Topology	92

9.4.3 Adaptive Algorithm Selection	92
--	----

List of Figures

1.1	Example of dense and sparse matrix structures. Sparse matrices contain far fewer nonzero elements.	2
1.2	Two different sparsity patterns commonly observed in real world matrices. The left matrix represents a social network, the right matrix is a stiffness matrix which represents a system of linear equations. . . .	4
1.3	The broadcast collective communication, showing that one process sends all of the data to other connected processes.	5
2.1	An example of the COO format, showing the underlying data and visual index associations.	8
2.2	An example of the CSR format, showing the underlying data and visual index associations.	9
2.3	An example of the CSC format, showing the underlying data and visual index associations.	11
2.4	An example of a uniformly random sparse matrix.	12
2.5	An example of a block sparsity matrix.	14
2.6	An example of a kronecker product matrix.	16

2.7	MST Broadcast on $p = 4$ nodes with root at Node $n = 0$. Node $n = 0$ sends x to Node $n = 3$ in Step $n = 1$, then both nodes fan out to the remaining two nodes in Step 2, achieving full coverage in $\lceil \log_2 p \rceil = 2$ rounds.	20
2.8	MST Scatter on $p = 4$ nodes with root at Node $n = 0$ holding $x = (x_0, x_1, x_2, x_3)$. Only the subvector belonging to each subnetwork is transmitted at each step, halving the message size at every level. . . .	21
2.9	MST Gather on $p = 4$ nodes, collecting subvectors into Node $n = 0$. Messages travel toward the root, with each intermediate node forwarding an increasingly complete subvector up the tree.	22
2.10	MST Reduce on $p = 4$ nodes with root at Node $n = 0$. Partial sums are accumulated bottom up: each node reduces within its half before forwarding the result toward the root, performing $\lceil \log_2 p \rceil$ additions in total.	23
2.11	An example of the CSC format, showing the underlying data and visual index associations.	25
2.12	An example of the CSC format, showing the underlying data and visual index associations.	25
2.13	A single step of the SUMMA algorithm given that $k = 1$, showing both row and column distribution to the process grid.	27
6.1	Sparse MST Collectives wall time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	57
6.2	Sparse MST Broadcast time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	58

6.3	Sparse MST Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	59
6.4	Sparse MST Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	60
6.5	Sparse MST Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	61
6.6	Sparse Bucket Collectives wall time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	62
6.7	Sparse Bucket All Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	63
6.8	Sparse Bucket All Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	63
6.9	Sparse Bucket Reduce Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	64
6.10	Sparse MST BCast time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	65
6.11	Sparse MST Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	66
6.12	Sparse MST Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	66
6.13	Sparse MST Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	67
6.14	Sparse Bucket All Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	67
6.15	Sparse Bucket All Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	68

6.16	Sparse Bucket Reduce Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096, p = 8$)	68
6.17		70
6.18		71
6.19		72
6.20		73
6.21		74
6.22		75
6.23		76

List of Tables

3.1	Memory cost per entry and fixed overhead for each storage format (float32 values, int32 indices, $N \times N$ matrix with nnz nonzeros). . . .	34
3.2	Memory consumption at selected sparsity levels for a 100×100 matrix. The most efficient format at each sparsity level is shown in bold . . .	35
3.3	Memory consumption at selected sparsity levels for a 1000×1000 matrix.	35
3.4	Memory consumption at selected sparsity levels for a 10000×10000 matrix.	36
3.5	Sparsity crossover points at which one format becomes more memory efficient than another, for three matrix sizes.	36

Abstract

Efficient collective communication is a key challenge for scaling applications on distributed memory architectures. Prior analyses have largely focused on dense matrix computations, where data often moves in predictable ways. However, many real-world problems are expressed in terms of graphs whose adjacency matrices are sparse. The structure and storage format of these matrices can introduce irregular communication patterns in distributed networks. This thesis looks into the study of how matrix size, sparsity pattern, and storage format affect the performance of parallel communication collectives and sparse matrix multiplication. Using the SUMMA algorithm as a baseline, we evaluate communication and computation costs across a range of matrix aspect ratios, sparsity patterns, and sparse storage formats, including CSR and CSC. Our experiments use both synthetic matrices with controlled density and structure, as well as Kronecker-generated graphs, and are implemented using collective communication primitives in OpenMPI. Furthermore, we look to determine the limitations imposed by matrix shape and sparsity on statistics such as memory bandwidth, communication overhead, and overall multiplication performance.

Chapter 1

Introduction

Many real world computational problems are naturally represented as a series of sparse matrices. In these matrices, the majority of entries are zero, and thus only a small subset of elements actually contain meaningful information. These matrices arise in various domains, including graph analysis, computer graphics, science simulations, network modeling, and machine learning [1]. In many of these applications, the sparse matrices represent the structure of some underlying system, such as the adjacency matrix of a graph.

Unlike dense matrices, where every element contributes equally to computation and communication, sparse matrices encode structural information about the data. For example, the adjacency matrix of a social network contains nonzero entries only where connections exist between users in the network. A basic example of a dense matrix and a sparse matrix side by side is visualized in Figure 1.1.

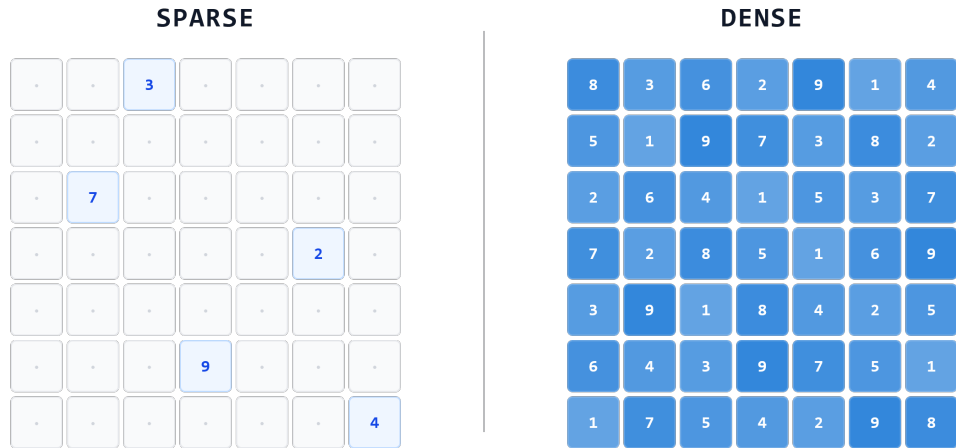


Figure 1.1: Example of dense and sparse matrix structures. Sparse matrices contain far fewer nonzero elements.

Because sparse matrices only store nonzero entries, they offer significant advantages in both memory usage and computational cost. However, the irregular nature of sparsity introduces challenges that do not appear in dense matrix computations, such as

1. Irregular memory access
2. Uneven distribution of elements
3. Additional metadata for representing these matrices
4. Unpredictable communication patterns in parallel systems

So algorithms designed for dense matrices cannot always be applied directly to sparse matrices. Understanding how the sparsity of a matrix, or the amount of nonzero values, interacts with algorithm design is a critical problem in modern day high performance computing.

1.1 Structure and Patterns

A key characteristic of sparse matrices is that their performance is not only determined by the number of nonzero entries, but also by how the entries are arranged. Two matrices may contain the same number of nonzero elements, however, due to their sparsity pattern may have drastically different performance characteristics. There are various common sparsity patterns that will be discussed in detail, but a few examples include:

1. Uniform Random Sparsity: Nonzero elements are distributed evenly across the matrix.
2. Banded Structures: Nonzero entries cluster near the diagonal of the matrix.
3. Block Structures: Nonzero entries form dense sub regions of the matrix.
4. Kronecker Product Matrices: Mimic the structure of many real world graphs through a repeated product of itself.

Figure [1.2](#) showcases two different sparsity structures, a social network and a fem stiffness matrix, side by side. The fem stiffness matrix also demonstrates a specific class of sparsity pattern, a banded matrix, which will be discussed in further detail later.

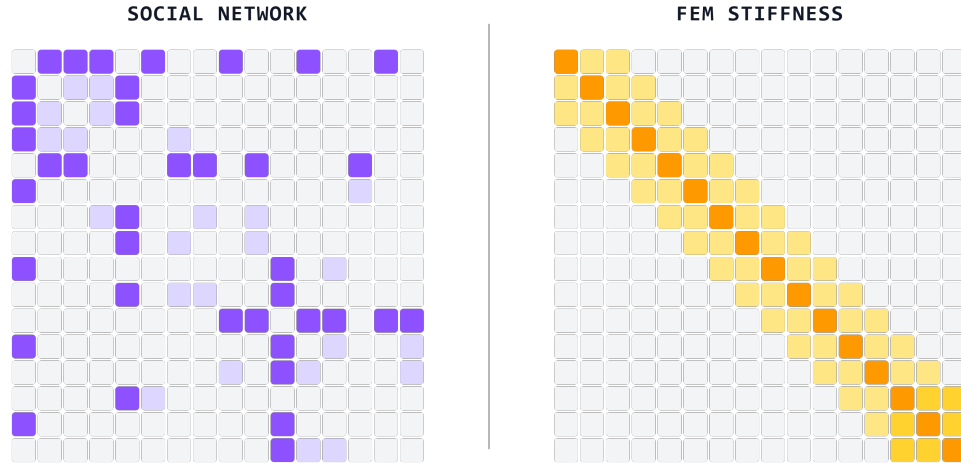


Figure 1.2: Two different sparsity patterns commonly observed in real world matrices. The left matrix represents a social network, the right matrix is a stiffness matrix which represents a system of linear equations.

1.2 Distributed Sparse Matrices

Many applications that use sparse matrices often operate at scales that exceed the capabilities of a single machine. With the ever increasing dependency on Artificial Intelligence, this is becoming the truth more than ever due to the extensive workloads that span across distributed networks of machines.

In these distributed systems, data is exchanged between processes to complete computations. These exchanges are typically implemented using collective communications, which are critical to coordinating communication across groups of processes and machines. Examples of this to be discussed in detail later include broadcast, all-gather, reduce-scatter, and all-reduce. Figure 1.3 showcases an example MPI Broadcast operation, where one process sends its data to all other processes in the connected network. For dense operations, these collective communications are relatively predictable because the data is often stored in contiguous arrays. Sparse matrices, however, introduce additional complexity. Since sparse matrices are stored with

only nonzero entries alongside the indexing metadata, communication often contains:

1. values and index information
2. irregular message sizes
3. data dependent communication patterns

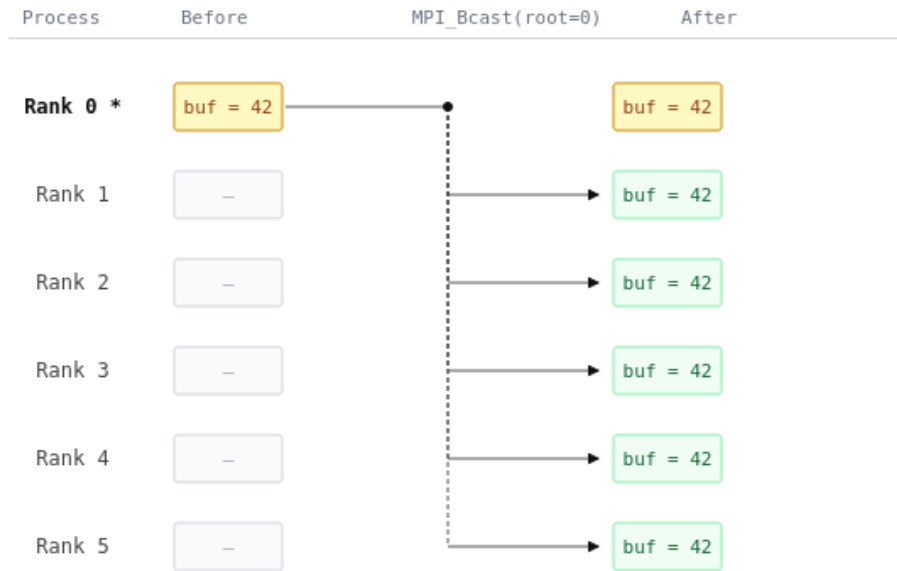


Figure 1.3: The broadcast collective communication, showing that one process sends all of the data to other connected processes.

1.3 Thesis Outline

This thesis will primarily consist of work on

1. Distributed dense and sparse matrices.
2. Collective communication operations as a measurement for communication cost for sparse algorithms.

3. Parallel matrix multiplication using the SUMMA algorithm to evaluate both computation and communication.
4. Matrices with controlled sparsity patterns.

And will be structured by building on sparse concepts from sparsity patterns to distributed matrix multiplication

1. Chapter 2 provides theoretical and practical context for sparse matrix formats, collective communications, and distributed algorithms.
2. Chapter 3 analyzes how collective communication operations behaves when applied to sparse data rather than dense arrays.
3. Chapter 4 extends the analysis to sparse SUMMA, examining how sparsity interacts with the algorithms two phase broadcast and multiply structure.
4. Chapter 5 describes the experimental methodology. It covers the matrix shapes and patterns used in testing, the storage formats and variants implemented, as well as the hardware and software environment and tools.
5. Chapter 6 presents and analyzes the experimental results. It evaluates the impact of sparsity pattern, sparsity shape, and storage format on both collective communication and SUMMA performance.
6. Chapter 7 examines the distributed data loading problem and presents some basic insights into what makes it particularly difficult.
7. Chapter 8 outlines the role of visualization in the communication of complex sparse and distributed behavior and problems.
8. Chapter 9 summarizes the contribution of this thesis and revisits previous work.

Chapter 2

Background

2.1 Sparse Matrix Representations

2.1.1 Coordinate (COO) Format

The Coordinate format is one of the simplest representations for sparse matrices [1]. In the COO format, each nonzero element is simply stored as a tuple containing its row index, column index, and value. Thus, the underlying structure for a COO Matrix may look like:

```
1 struct COOMatrix {  
2     int rows, cols, nnz;  
3     std::vector<int> row_indices;  
4     std::vector<int> col_indices;  
5     std::vector<float> values;  
6 };
```

Listing 1: An example COO representation in C++.

This structure is easy to construct and is commonly used as an intermediate format when building sparse matrices [1, 2]. However, the COO format does not support

any efficient random access or row/column traversal due to the nature of how the data is stored, and thus is not often used for computations and is converted to other formats such as CSR or CSC. Figure 2.1 shows an example of a COO matrix and the individual mapping to the lower level arrays.

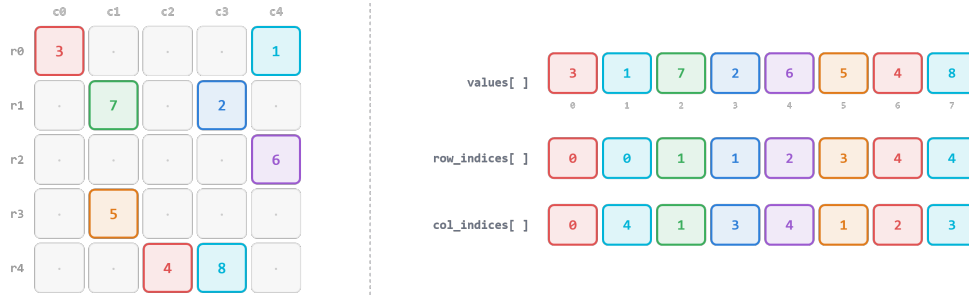


Figure 2.1: An example of the COO format, showing the underlying data and visual index associations.

2.1.2 Compressed Sparse Row

The Compressed Sparse Row (CSR) format is one of the most widely used sparse matrix representations in high performance computing [1]. In the CSR format, each nonzero element is stored as a tuple containing its value, column index, and compressed row index. Thus, the underlying structure for a CSR Matrix may look like:

```

1 struct CSRMatrix {
2     int rows, cols, nnz;
3     std::vector<int> row_ptrs; // length rows + 1
4     std::vector<int> col_indices; // length nnz
5     std::vector<double> values; // length nnz
6 };

```

Listing 2: An example CSR representation in C++.

The main compression outside of only storing non zero values comes from the

row_ptrs array. Instead of storing a row index for every nonzero element like the COO format, CSR only stores the starting offset for each row. This reduces the row index storage from $O(nnz)$ to $O(m)$ which becomes a significant factor as $nnz > m$. Retrieving all nonzero elements in some row i only requires a slice of values from *row_ptrs*[i] to *row_ptrs*[$i + 1$] and thus row traversal is simply a $O(nnz_i)$ operation where nnz_i is the number of nonzeros in that row. An example of this format can be seen in Figure 2.2 which illustrates the mapping between the visual structure of a matrix and the three underlying arrays.

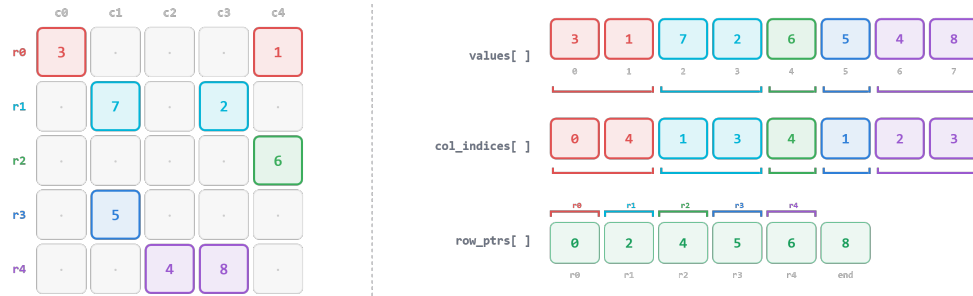


Figure 2.2: An example of the CSR format, showing the underlying data and visual index associations.

The row oriented access pattern makes CSR well suited for operations that proceed row by row, such as sparse matrix vector multiplication (SpMV) and the row broadcasts that will be discussed later in the SUMMA algorithm. Conversely, random access to individual elements requires a binary search within the *col_indices* array for a given row and thus is a $O(\log(nnz_i))$ operation. As a result of this, CSR is typically only used for computational problems rather than constructing or modifying sparse matrices in place.

2.1.3 Compressed Sparse Column

The Compressed Sparse Column (CSC) format is the column major variant of the CSR format. The underlying arrays are identical in concept but transposed to compress the columns rather than the rows, as demonstrated below:

```
1 struct CSCMatrix {  
2     int rows, cols, nnz;  
3     std::vector<int> col_ptrs; // length cols + 1  
4     std::vector<int> row_indices; // length nnz  
5     std::vector<double> values; // length nnz  
6 };
```

Listing 3: An example CSC representation in C++.

The CSC format follows all of the same computational behavior as CSR but with respect to columns instead of rows and thus makes CSC the natural format to be used when an algorithm needs to iterate over columns. This is similarly useful for the soon to be discussed SUMMA algorithm as there are points within the algorithm that require columns to be broadcast down the process grid and using the CSC format makes that a contiguous scatter operation.

CSC is also found in sparse matrices that arise from graph problems when adjacency is expressed column wise. For instance, in certain variants of graph neural networks or PageRank where incoming edge lists are column indexed [3, 4]. Figure 2.12 shows an example of the format visually.

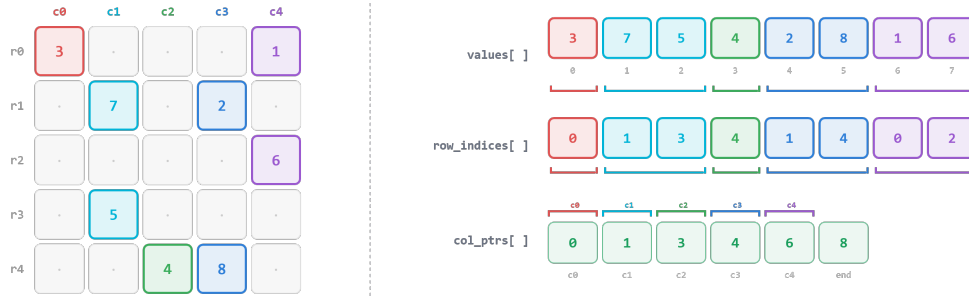


Figure 2.3: An example of the CSC format, showing the underlying data and visual index associations.

2.2 Sparsity Patterns and Matrix Structure

As established in Chapter 1, two sparse matrices with the same number of nonzero elements can exhibit very different performance characteristics depending on where those elements are located. This distribution of nonzero entries is referred to as the sparsity pattern. This section describes many of the major sparsity patterns encountered in practice and discusses how each arises in the real world and what computational challenges they introduce.

2.2.1 Uniform Random Sparsity

Uniform Random Sparsity is the simplest sparsity pattern and serves as a good baseline against which all other patterns can be compared. In a uniformly random sparse matrix, every entry is independently nonzero with some probability p , where p is the target density of the matrix. This produces a sparsity pattern in which there is no regularity as nonzero entries are statistically scattered evenly across rows and columns. So, there should be no clustering, banding, or hierarchical structure. Figure 2.4 visualizes an example of a uniformly random sparse matrix.

SPARSE MATRIX

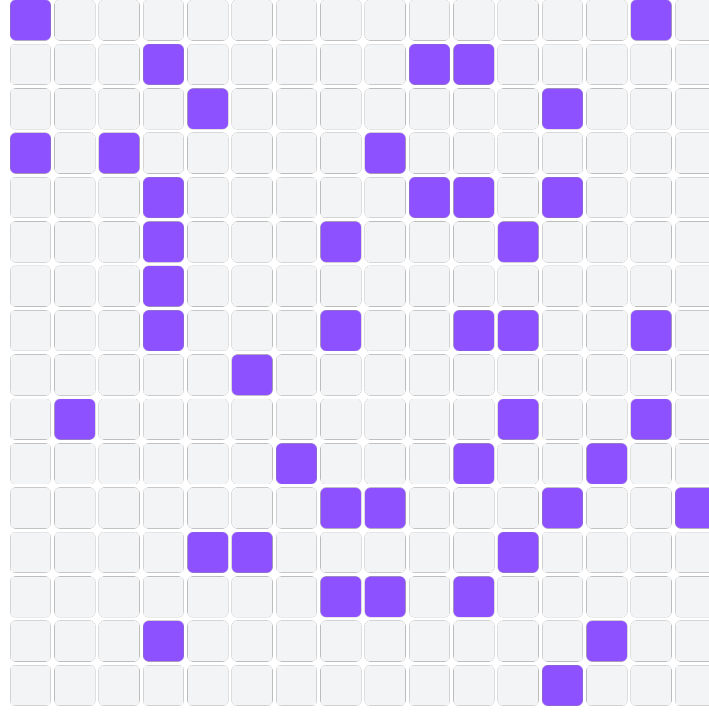


Figure 2.4: An example of a uniformly random sparse matrix.

In practice, truly uniform random sparsity is rare in real world data. Its primary utility, especially in this thesis, is used as a controlled experimental condition. Because the expected number of nonzeros per row and column is constant and essentially equal to $p \times dimension$, a uniformly random sparse matrix distributes the load evenly across processes and avoids any imbalance effects. Similarly for collective communications, uniformly random sparsity presents the most predictable case. Since each process receives approximately the same amount of nonzero entries, variance in message sizes and thus costs are low relative to structured patterns. And so it can be most similarly applied to the dense case as it will have nearly uniform scaling.

2.2.2 Banded Sparsity

Banded sparsity can be characterized by nonzero entries that are concentrated near the main diagonal of the matrix. Banded matrices arise naturally from the discretizations of differential equations over regular grids. For example, a one dimensional finite difference discretization of the Poisson equation produces a tridiagonal matrix, while a two dimensional discretization on an $n \times n$ grid produces a matrix with a band width of approximately n [1, 5]. The fem stiffness matrix shown in Figure 1.2 is an example of this sparse matrix pattern.

Compared to Uniform Sparse Matrices, banded sparsity introduces a number of favorable properties when it comes to computation and communication. Both row wise and column wise access, depending on the format the matrix is stored in, can be efficient as the non zero entries would be stored in contiguous index space. For example, when a banded sparsity matrix is stored in the CSR format it would be efficient to access entire rows of nonzero entries as they are stored contiguously. Conversely, trying to access columns of data in a banded matrix stored in CSR format would be extremely inefficient. This also introduces natural load balancing as it would be efficient to send bands of the matrix at a time. Because both CSR and CSC provide efficient access to banded matrices, the choice between them will be driven by the access pattern of the algorithm consuming the matrix, and in particular whether or not it is column wise or row wise.

2.2.3 Block Sparsity

Block sparsity refers to matrices in which nonzero entries are organized into dense rectangular sub blocks. This structure is commonly observed in graph problems where nodes represent collections of entities and in some neural network weight matrices.

Figure 2.5 visualizes an example of a sparse matrix with a block sparsity pattern.

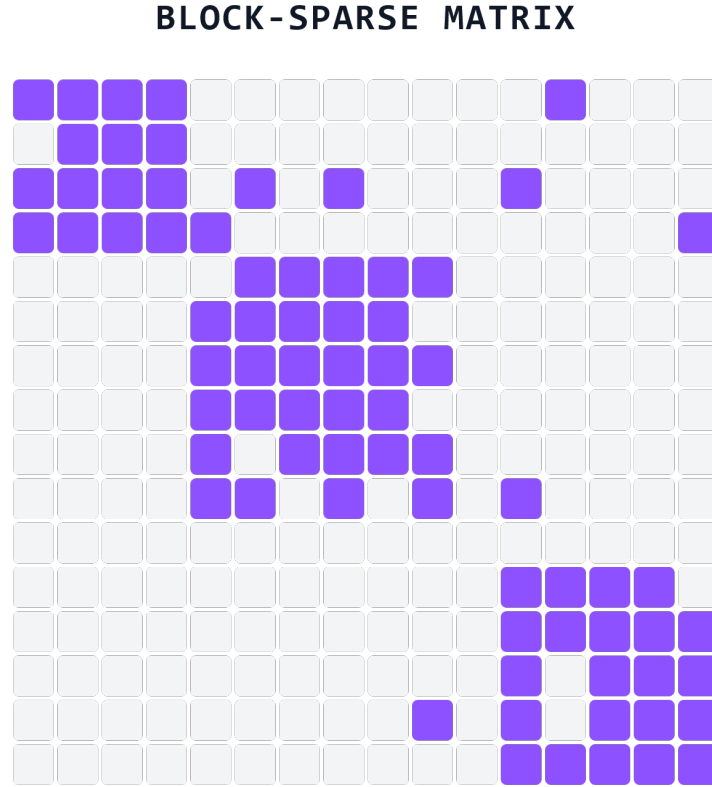


Figure 2.5: An example of a block sparsity matrix.

In practice, block sparsity offers several computational advantages over unstructured patterns. Because non zero entries are concentrated into dense rectangular regions, the local computation within each block can be handled by optimized BLAS kernels rather than general sparse routines [6]. This allows these algorithms to exploit both hardware level parallelism and the cache in ways that unstructured patterns cannot. Additionally, while not covered in this thesis, specialized formats like BCSR (Block Compressed Sparse Row) which extends CSR by grouping non zeroes into fixed blocks can also be used which reduce metadata overhead and allow for better memory access patterns [7].

For collective communications, block sparsity is advantageous as it partially helps

with the message size predictability problem. When blocks align with the process boundaries, the number of non zeroes per process is more accurately determined by the block dimensions and thus can make buffer preallocation straightforward. The tradeoff however is that if blocks are not aligned carefully with the process grid than it can cause them to be split which destroys any advantages this solution created, and thus process grid alignment is an important consideration when using this format.

2.2.4 Kronecker Product Sparsity

A Kronecker product matrix is constructed by repeatedly applying the Kronecker product to a smaller initiator matrix [8, 9]. Given some initiator matrices A of size $m \times n$ and B of size $p \times q$, their Kronecker product $A \otimes B$ is a matrix of size $mp \times nq$ in which each entry a_{ij} of A is replaced by the scaled block $a_{ij} \times B$. Repeated application of this operation thus produces large matrices with a similar, hierarchical sparsity structure, as visualized in Figure 2.6.

KRONECKER MATRIX

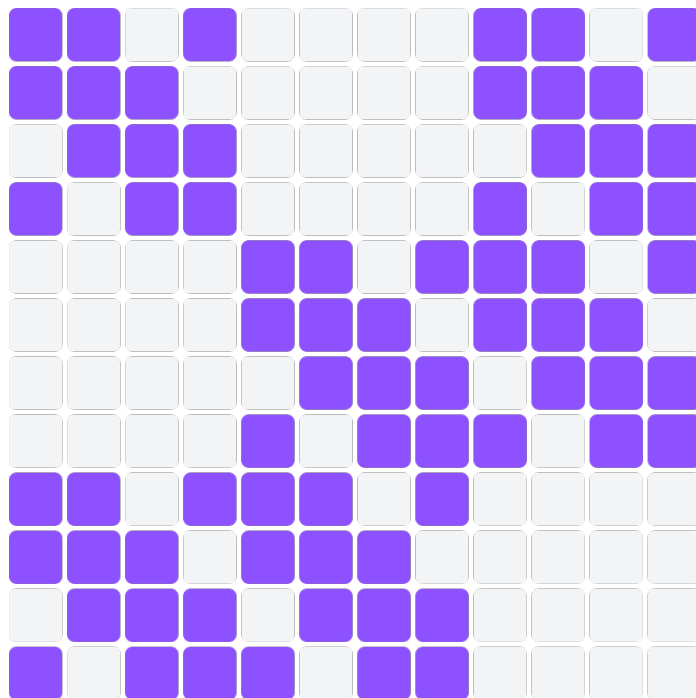


Figure 2.6: An example of a kronecker product matrix.

Kronecker product matrices are widely used in graph benchmarking, most notably in the Graph500 benchmark [10], because the resulting adjacency matrices present properties similar to real world graphs such as: skewed degree distributions and clustering. These properties make Kronecker graphs a more realistic and insightful test case as compared to uniformly random matrices. However, the same structure that makes them realistic also causes a significant load imbalance in our distributed setting since portions of the rows and columns will have far fewer non zero entries than the rest. This is discussed further in Chapter 3.

2.3 Distributed Memory Parallelism (MPI)

As noted in Chapter 1, in the modern age of computing single core algorithms are a relic of the past with modern CPUs being capable of performing hundreds of operations in parallel and GPUs being able to perform millions of operations in parallel (TODO CITE). Additionally, many modern computational problems are often distributed across dozens of individual machines (nodes or ranks) which in and of themselves may contain multiple instances of a program working on a problem. This type of distributed memory system is critical for problems where a single machines worth of memory is physically impractical as problem sizes reach requirements in the hundreds of gigabytes.

The Message Passing Interface (MPI) is the standard API for programming these distributed memory systems [11]. MPI provides a portable, vendor neutral specification for sending and receiving messages between processes as well as an extensive set of collective operations that coordinate communication across groups of processes. OpenMPI is a widely used open source implementation of the MPI standard that provides high performance communication[12]. All distributed experiments in this thesis are implemented using OpenMPI.

In an MPI program, processes are identified by their rank within a communicator, which is a group of processes that can communicate with one another. A global communicator, `MPI_COMM_WORLD`, is defined as all processes that are launched by the MPI runtime. Custom communicators are available to be created for problems that see communication across common sets of processes, such as when rows and columns are assigned to individual processes and need to be distributed. Processes in the MPI runtime interact directly via a collection of `MPI_Send` and `MPI_Recv` operations, often wrapped as a set of collective communications. The simple point

to point example in Listing 4 illustrates an example of Rank 0 sending an integer to Rank 1.

```
1  // Fetch the local instances rank and the total world size
2  int rank, size;
3  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
4  MPI_Comm_size(MPI_COMM_WORLD, &size);
5
6  int value = 0;
7  if (rank == 0) {
8      value = 42;
9      MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
10     std::cout << "Process 0 sent value " << value << std::endl;
11     // "Process 0 sent value 42"
12 }
13 else if (rank == 1) {
14     MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
15     std::cout << "Process 1 received value " << value << std::endl;
16     // "Process 1 received value 42"
17 }
```

Listing 4: Simple point to point communication using OpenMPI

2.4 Collective Communications

As noted previously, Collective Communication operations are designed to get a group of nodes to consolidate or distribute data. The operations are typically divided into

two main categories: data redistribution operations (broadcast, scatter, gather, all gather) which move data around without any arithmetic, and data consolidation operations (reduce, reduce-scatter, all reduce) which combine contributions from all nodes using operations like summation. It should be noted that this is not an exhaustive list of all collective communications, but a subset that is discussed in this thesis.

The cost of any of these collectives is analyzed under the $\alpha - \beta$ model, where sending some message of n items costs $\alpha + n\beta$ [13]. The term α is indicative of the per message start latency and β is the communication time per item. For reduction operations, there is also an extra term γ which is an additional per item compute cost.

Adapting the following collectives to operate on sparse formats introduces a completely different cost analysis which completely depends on the sparsity pattern of these matrices. Without a known sparsity pattern, doing static analysis of the cost is unpredictable. Implementation of these algorithms in the sparse world will be discussed further in Chapter 3.

2.4.1 MST Broadcast, Scatter, Gather, and Reduce

The minimum spanning tree (MST) algorithms are the standard for short vector algorithms [13] for these four operations. The latency lower bound $\lceil \log_2 p \rceil \alpha$ as noted by Chan is achieved by halving the number of nodes that need to be communicated at every step. The main algorithm is a recursive partition such that at each step, p nodes are split into a left half $\{0, \dots, m\}$ and a right half $\{m + 1, \dots, p - 1\}$ where $m = \lfloor p/2 \rfloor$. A root node from each half is chosen and a single message is exchanged across the two roots. This algorithm continues recursively choosing new roots until the operation is complete. Because the two separate halves proceed in parallel, the

max depth of the communication is $\lceil \log_2 p \rceil$.

2.4.1.1 Broadcast

In the MST Broadcast operation, the root node holds some vector x that must be sent to every other node. At each level of recursion, the current root node sends the full vector to a node in the other half, after which both halves recursively perform that same operation. As shown in Figure 2.7, with a root of $n = 0$ and $p = 4$, the first step sends x from $n = 0$ to $n = 3$. The next step sends x simultaneously from $n = 0$ to $n = 1$ and from $n = 3$ to $n = 2$.

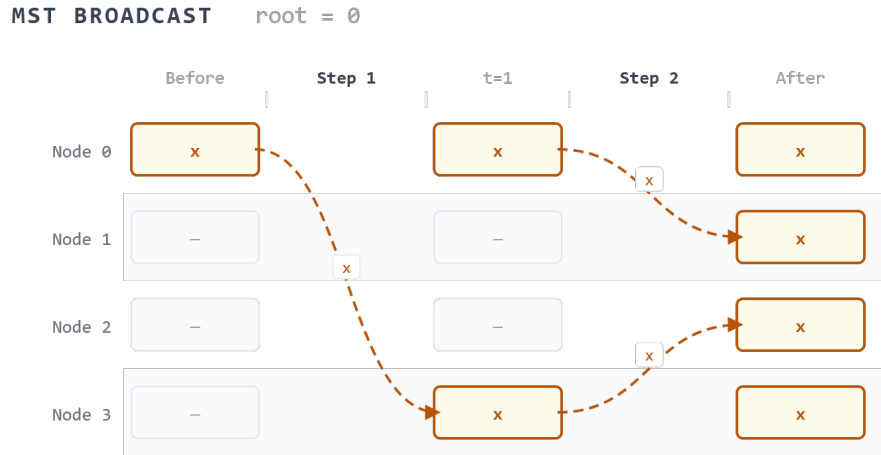


Figure 2.7: MST Broadcast on $p = 4$ nodes with root at Node $n = 0$. Node $n = 0$ sends x to Node $n = 3$ in Step $n = 1$, then both nodes fan out to the remaining two nodes in Step 2, achieving full coverage in $\lceil \log_2 p \rceil = 2$ rounds.

2.4.1.2 Scatter

The MST Scatter operation is a simple modification of a MST Broadcast, in which the root only sends a portion of the data to the receiving node rather than the entire vector. This is shown in Figure 2.8: Node $n = 0$ sends x_2, x_3 to $n = 3$ in Step 2, then

sends x_1 to $n = 1$ while Node $n = 3$ sends x_2 to Node $n = 2$.

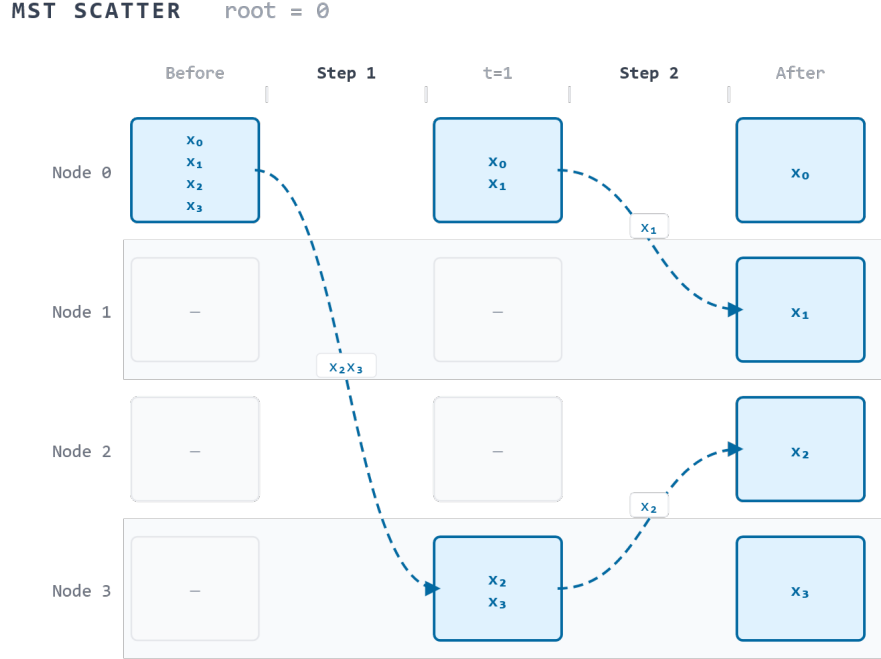


Figure 2.8: MST Scatter on $p = 4$ nodes with root at Node $n = 0$ holding $x = (x_0, x_1, x_2, x_3)$. Only the subvector belonging to each subnetwork is transmitted at each step, halving the message size at every level.

2.4.1.3 Gather

The MST Gather operation is simply the inverse of the MST Scatter operation, such that after every step the amount of elements increases as the recursion reaches the target root node. Figure 2.9 shows that Node $n = 1$ sends x_1 to Node $n = 0$ and Node $n = 2$ sends x_2 to Node $n = 3$ in Step 1, then Node $n = 3$ sends the combined x_2, x_3 to Node $n = 0$ in Step 2.

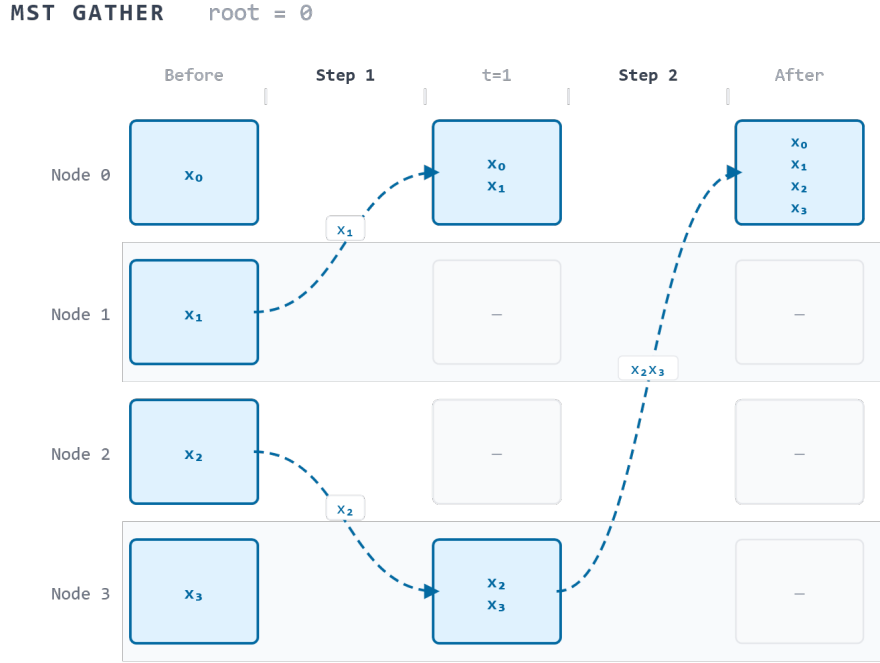


Figure 2.9: MST Gather on $p = 4$ nodes, collecting subvectors into Node $n = 0$. Messages travel toward the root, with each intermediate node forwarding an increasingly complete subvector up the tree.

2.4.1.4 Reduce

Finally the MST Reduce operation is a modification of an MST Broadcast. Whenever a node receives some value x , it will perform an operation such as summation on the incoming x with its own value, and then broadcast that value towards the other halves root node. Figure 2.10 in Step 1 has Node $n = 1$ send $x^{(1)}$ to Node $n = 0$, and Node $n = 3$ send $x^{(3)}$ to Node $n = 2$, forming two partial sums; Step 2 has Node $n = 2$ send the partial sum $x^{(2)} + x^{(3)}$ to Node $n = 0$, which adds it to its own partial sum to obtain the total. As previously mentioned, due to the addition of a computational cost an additional γ term is added to the total operation cost.

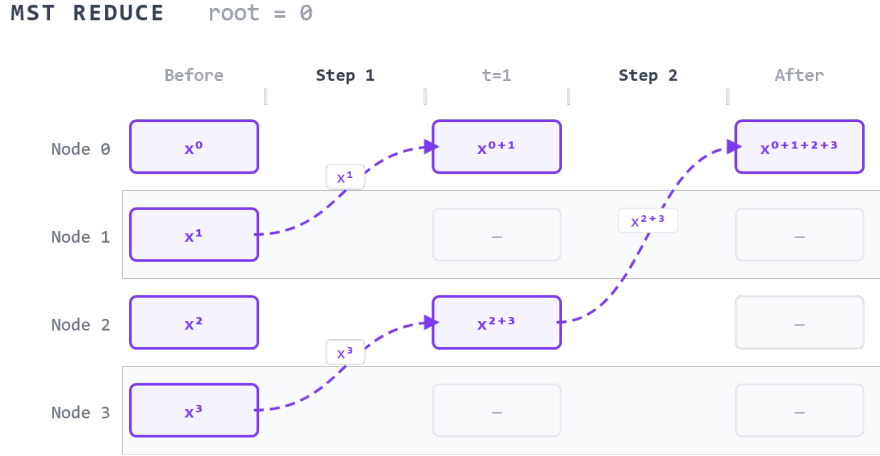


Figure 2.10: MST Reduce on $p = 4$ nodes with root at Node $n = 0$. Partial sums are accumulated bottom up: each node reduces within its half before forwarding the result toward the root, performing $\lceil \log_2 p \rceil$ additions in total.

2.4.2 Bucket All Gather and Reduce Scatter

The bucket algorithms are the standard approach for bandwidth optimal implementations of the All Gather and Reduce Scatter collectives. Unlike the MST algorithms which prioritize achieving optimal latency by minimizing the communication rounds, the bucket algorithms focus on minimizing the total volume of data transmitted per process. This matters a lot whenever the message size is large, as for example the MST broadcast operation sends the full message along every edge of the spanning tree, while the bucket approach routes each chunk of data once.

In both the bucket All Gather and Reduce Scatter collectives, all p processes are arranged logically in a ring topology, where the algorithm proceeds in $p - 1$ steps. In each step, each process simultaneously sends data to its right neighbor and receives data from its left neighbor. Each message thus contains at most $\frac{1}{p}$ of the total data, so the per step communication volume is $\frac{n}{p}$ items.

The Reduce Scatter operation is equivalent to performing a distributed reduction such that after the algorithm completes, each process holds the reduced result for its assigned $\frac{1}{p}$ chunk of the output data.

The All Gather is the inverse of the Reduce Scatter collective. Each process begins with some distinct $\frac{1}{p}$ chunk of the final result and must receive all other chunks. In each step, every process forwards the most recently received chunk clockwise around the ring, such that after $p - 1$ steps every process has received every chunk of the total data.

For sparse data, the bucket algorithms present a challenge. Because each step transmits a fixed $\frac{1}{p}$ slice of the data, the slicing must be defined in terms of the sparse structure. For example, if the data is partitioned by rows then each step transmits approximately $\frac{nnz}{p}$ nonzero values, but is still fully dependent on how the nonzero entries are distributed across row slices. A highly skewed sparsity pattern can cause some steps to transmit for more data than others, destroying the balance that makes the bucket algorithm communication efficient.

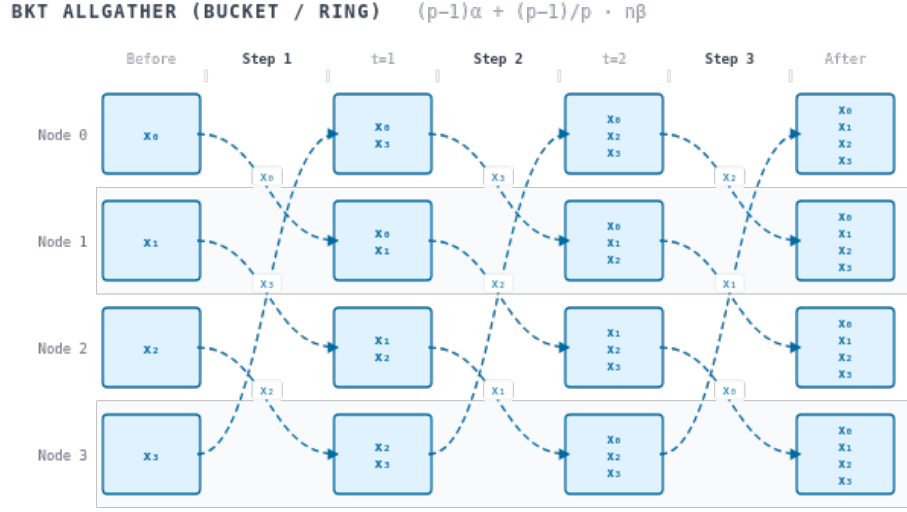


Figure 2.11: An example of the CSC format, showing the underlying data and visual index associations.

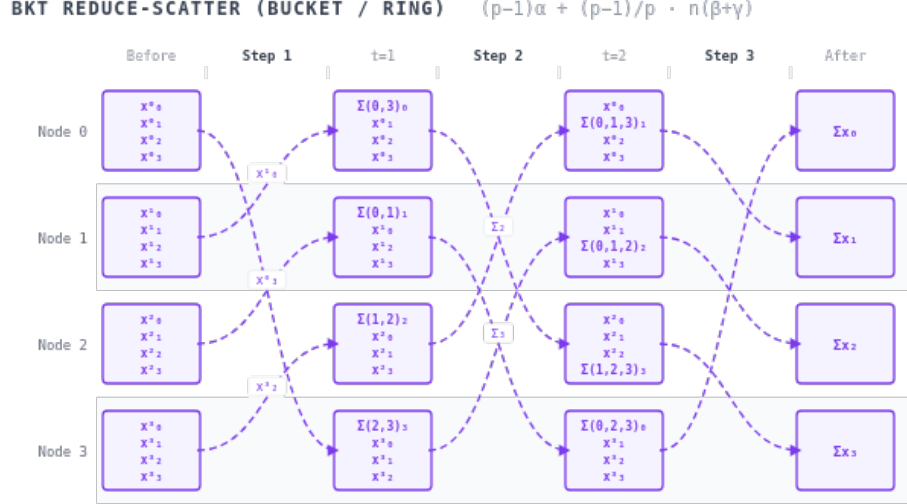


Figure 2.12: An example of the CSC format, showing the underlying data and visual index associations.

2.5 Distributed Matrix Multiplication (SUMMA)

The Scalable Universal Matrix Multiplication Algorithm (SUMMA) is a distributed matrix multiplication algorithm designed for two dimensional process grids [14]. Given a matrix multiplication $C = A \times B$ where A is $m \times k$, and B is $k \times n$, and C is $m \times n$, SUMMA distributed all three matrices across some $P_r \times P_c$ process grid. Each process (i, j) owns a local block of each matrix and proceeds by processing blocks of size b .

Choosing a two dimension process grid is an important part of SUMMA's design. A one dimension process grid would cause entire rows of A and C to be assigned to each process, which in turn requires every process to hold an $O(n^2)$ copy of B which eliminates any memory benefits. For a two dimension grid, this is solved because both input matrices are split across the process grid and thus requires processes to only hold no more than $O(n^2/P)$ memory.

The SUMMA algorithm operates in k/b steps, where b is the panel block size. At each step s , there are a few operations that are performed:

1. The process in column s of the grid broadcasts its local block of A along the entire process row, such that every process in that row receives the A panel of the column.
2. The process in row s of the the grid broadcasts its local block of B along its entire process column, such that every process in the column receives the B panel of the row.
3. Each procoess computes the local outer product contribution $C_{ij+} = A_{local} \times B_{local}$ accumulating into its local block of C .

Figure 2.13 demonstrates a single step of SUMMA where $k = 1$. The pseudocode

in Listing 5 shows the corresponding OpenMPI implementation using MPI_BCast over individual row and column communicators for dense matrices.

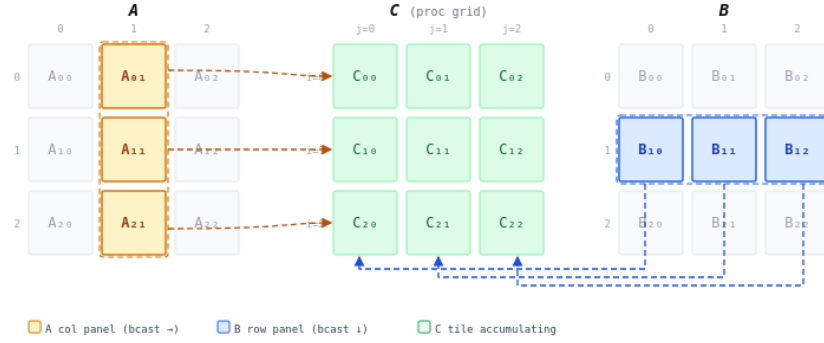


Figure 2.13: A single step of the SUMMA algorithm given that $k = 1$, showing both row and column distribution to the process grid.

2.5.1 Block Size Tradeoffs

One important note is the tradeoffs that come from adjusting the block size b . Smaller values of b increase the number of outer loop iterations to k/b , where the per step latency cost is then multiplied by the same factor (essentially, as b decreases the problem becomes more latency bound). In contrast, larger values of b increase the latency per step and allows the local matrix multiply to work on larger panels of b which can offset the load to the local multiplication kernel. However, a larger b also means the amount of memory required to operate is scaled. In practice for dense matrices, the analysis of the block size for b has been studied for whatever achieves peak performance [14]. For sparse matrices, this is complicated as blocks of size b may not always contain the same number of non zero entries based on the sparsity pattern.

```

1  // Assume a 2D process grid:  $P_r \times P_c$ 
2  // Each process holds local blocks  $A_{ij}$ ,  $B_{ij}$ ,  $C_{ij}$ 
3
4  // Create 2D Cartesian communicator
5  MPI_Comm grid;
6  MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &grid);
7
8  // Determine row and column communicators
9  MPI_Comm row_comm, col_comm;
10 MPI_Comm_split(grid, row_coord, col_coord, &row_comm);
11 MPI_Comm_split(grid, col_coord, row_coord, &col_comm);
12
13 // Initialize local matrices
14 Matrix A_local, B_local, C_local;
15 for (k = 0; k < num_blocks; k++) {
16     // Broadcast A block across process row
17     if (col_coord == k)
18     {
19         A_panel = A_local;
20     }
21     MPI_Bcast(A_panel, ..., root=k, row_comm);
22
23     // Broadcast B block down process column
24     if (row_coord == k)
25     {
26         B_panel = B_local;
27     }
28     MPI_Bcast(B_panel, ..., root=k, col_comm);
29
30     // Local matrix multiply
31     C_local += A_panel * B_panel;
32 }

```

Listing 5: Pseudocode for SUMMA matrix multiplication using OpenMPI

2.5.2 Communication Cost

For dense matrices, the communication cost of SUMMA is well studied. Each of the k/b outer steps performs two MPI_Bcast operations: one over the process row of P_c ranks and one over the process column of P_r ranks. Under the $\alpha - \beta$ model,

broadcasting a message of some size n over p ranks using the MST algorithm costs $\log_2 p \alpha + n \beta$. Over all of the steps, the total volume of communication can be described as $O(mn/P_c + kn/P_r)$.

The overall communication cost given above is a simplification, and can be expanded out to the following.

$$T_{comm} = (k/b) \times (\lceil \log_2 P_c \rceil \alpha + (mb/P_r) \beta + \lceil \log_2 P_r \rceil \alpha + (nb/P_c) \beta)$$

Adapting the SUMMA algorithm for sparse matrices complicates the algorithm at practically every step but introduces many new perspectives on the analysis of effects of sparsity patterns. The implementation of sparse SUMMA will be discussed further in Chapter 4.

Chapter 3

Sparse Collective Communications

This chapter analyzes how the seven collective communication operations introduced in Chapter 2 behave when their payloads are sparse matrices rather than dense arrays. The goal here is not to report measurements or experiments but to build an understanding of where and why sparse data changes the cost structure of each collective.

3.1 Adapting Collectives to Sparse Data

Collective communication algorithms are inherently designed around the assumption that the message payload is some continuous array of fixed size elements, and thus every process in the communicator contributes or receives the same amount of data. Sparse matrices of course violate this assumption. Adapting collectives to sparse data therefore requires addressing three problems that are not typically present: variable message sizes, index metadata overhead, and data dependent communication patterns.

3.1.1 Variable Message Sizes

For dense collectives, the size of every message at every step of the algorithm is known statically from the matrix dimensions and the process count. For example, a broadcast of a $m \times n$ matrix over p processes always moves exactly $m \times n \times \text{sizeof}(\text{data})$ bytes. However, for a sparse matrix the message size depends on the number of nonzero entries in the region being communicated, which may vary between matrices, across steps of an algorithm, and across processes. This presents two main issues for sparse collectives.

First, the receiving process cannot preallocate buffers of the correct size without first being told how many nonzero values to expect. This adds an additional metadata exchange, although only typically a small integer, that must be sent ahead of the data. And then, between when it receives that metadata exchange and the full set of data it must allocate enough space for the incoming data. For short vector collectives where the main cost already comes from latency rather than bandwidth, this metadata exchange is a meaningful statistic that can impact performance. Secondly, standard MPI collective primitives such as `MPI_Bcast` and `MPI_Scatter` assume fixed message sizes across all processes. Sparse collectives must therefore use different variants that support variable counts for processes, or do additional processing before collectives can be used.

3.1.2 Metadata Overhead

A dense matrix can be transmitted simply as a single array of values because the position of each element is implicit from its offset in the buffer. A sparse matrix however must transmit additional information alongside values so that the receiver can construct the matrix. The cost of this metadata depends on the storage format.

The COO format transmits two full integer arrays alongside the values, the row_indices and col_indices, each of which are a length of nnz . The total message size for a COO matrix with nnz nonzeros can therefore be calculated as $(2 \times nnz \times sizeof(dimension)) + (nnz \times sizeof(data))$ where $sizeof(dimension)$ is the size of the metadata arrays (typically an integer), and $sizeof(data)$ is the size of the values data type. CSR transmits a col_indices of length nnz and a row_ptrs of length $m + 1$ whose total message size can be calculated as $nnz \times sizeof(dimension) + (m + 1) \times sizeof(dimension) + nnz \times sizeof(data)$. CSC is symmetric to CSR with col_ptrs of length $n + 1$ replacing row_ptrs.

The implication here is that COO always transmits more metadata than CSR and CSC for the same matrix, regardless of sparsity pattern or shape. However, both CSC and CSR can encounter scenarios where the matrix pattern does not allow for maximum compression to be achieved. For example, when using CSR with taller matrices where $m \gg nnz$ the rows_ptr overhead can be quite significant.

3.2 Data Dependent Patterns

In dense collectives, the communication schedule is determined entirely by the algorithm and the process count. The data itself has no influence on the communication pattern. In sparse collectives, the data influences the communication in two ways.

For scatter and gather operations, the amount of data sent or received to a process depends entirely on how many nonzeros fall within that process's assigned partition. A perfectly uniform sparsity distribution would give each process an equal share, however, real sparsity patterns are rarely uniform. For example, banded matrices distribute nonzeros evenly across rows but unevenly if the band does not align with the partition boundaries. This data dependent load imbalance means the slowest

process in the communicator determines the completion time of the collective and thus the degree of imbalance is tied directly to the sparsity pattern.

On top of that, the storage format also determines how efficiently the initial process can construct the message for each recipient. For a row partitioned scatter, the CSR format can easily extract the rows assigned to each process in $O(1)$ index lookups followed by fast contiguous memory copies. COO on the other hand must scan all nnz entries to identify which belong to each partition and thus costs $O(nnz)$ per partition. Following this row partitioned scatter, the CSC format must reconstruct these rows from a column oriented structure, requiring a full scan across all columns. So, the format of these matrices not only effects the size of a message but also the time it takes to construct it and reconstruct it, contributing to the total observed communication cost.

3.3 Communication Cost Analysis

This section is going to continue upon the cost model introduced in Chapter 2 to account for the overhead introduced by sparse data. For each collective, we will approximate a cost function based on the number of non zeroes nnz , the matrix dimensions $m \times n$, the process count p , and the storage format.

3.3.1 Cost Model for Sparse Data

For any message, we can define its size in bytes as $S(format, nnz, m, n)$ that is required to broadcast a sparse matrix of size nnz with dimensions $m \times n$ in some given format. Let $s_v = sizeof(data)$ and $s_d = sizeof(dimension)$:

1. $S_{COO} = nnz \times (2s_d + s_v)$

$$2. S_{CSR} = nnz \times (s_d + s_v) + (m + 1) \times s_d$$

$$3. S_{CSC} = nnz \times (s_d + s_v) + (n + 1) \times s_d$$

3.3.2 Sparsity Level vs Volume

The density of a matrix, defined as $\frac{nnz}{m \times n}$ is the primary determinant of raw communication volume for sparse collectives. As the density approaches one, a sparse format will approach or exceed the size of the equivalent dense representation due to the added metadata overhead. For example, a CSR matrix with some size nnz requires $nnz \times (s_{data} + s_{dimension} + (m + 1) \times s_{dimension})$ bytes, compared to $m \times n \times s_{data}$ for a dense matrix. This means the crossover point at which sparse storage offers no size advantage, and in fact incurs worse communication costs, occurs at a sparsity of $1 - s_{dimension} / (s_{data} + s_{dimension})$, which for floating point values and 32 bit integers (the basis of all experiments ran in this thesis) is roughly 50 percent sparsity.

Table 3.1: Memory cost per entry and fixed overhead for each storage format (float32 values, int32 indices, $N \times N$ matrix with nnz nonzeros).

Format	Bytes / entry	Fixed overhead	Total formula
Dense	4 B / element	none	$4N^2$
COO	12 B / nnz	none	$12 \cdot nnz$
CSR/CSC	8 B / nnz	$4(N + 1)$ B	$8 \cdot nnz + 4(N + 1)$

For the communication patterns studied here, sparsity operates as a continuous scaling factor on message sizes. Halving the sparsity approximately halves the communication volume for fixed matrix dimensions, which translates directly to reduced bandwidth cost. However, the latency cost is not reduced proportionally as the number of messages and communication rounds is determined by the algorithm and process count. At very low densities, sparse collectives are therefore prone to latency rather than bandwidth and thus the α term becomes the issue as we scale.

Table 3.2: Memory consumption at selected sparsity levels for a 100×100 matrix. The most efficient format at each sparsity level is shown in **bold**.

Sparsity (%)	nnz	Dense	COO	CSR/CSC	Winner
0.0	10 000	39.06 KB	117.19 KB	78.52 KB	Dense
50.0	5000	39.06 KB	58.59 KB	39.46 KB	Dense
75.0	2500	39.06 KB	29.30 KB	19.93 KB	CSR/CSC
90.0	1000	39.06 KB	11.72 KB	8.21 KB	CSR/CSC
95.0	500	39.06 KB	5.86 KB	4.30 KB	CSR/CSC
99.0	100	39.06 KB	1.17 KB	1.18 KB	COO

Table 3.3: Memory consumption at selected sparsity levels for a 1000×1000 matrix.

Sparsity (%)	nnz	Dense	COO	CSR/CSC	Winner
0.0	1 000 000	3.81 MB	11.44 MB	7.63 MB	Dense
50.0	500 000	3.81 MB	5.72 MB	3.82 MB	Dense
75.0	250 000	3.81 MB	2.86 MB	1.91 MB	CSR/CSC
90.0	100 000	3.81 MB	1.14 MB	785.16 KB	CSR/CSC
95.0	50 000	3.81 MB	585.94 KB	394.54 KB	CSR/CSC
99.0	10 000	3.81 MB	117.19 KB	82.04 KB	CSR/CSC

3.3.3 Load Imbalance across Processes

Load imbalance is created whenever nonzero entries are not distributed uniformly across process partitions. For scatter and gather operations in particular, the collectives wall time is bounded by the most heavily loaded process and thus all other processes must wait for the largest message to complete before the operation can finish. So, the amount of imbalance is a direct product of the sparsity pattern.

For a matrix with nnz non zeroes distributed across p processes, we can note that nnz_i for some process i . The ideal case is that $nnz_i = nnz/p$ for all i , which is statistically achieved by uniform random sparsity. An imbalance ratio given by $\max_i(nnz_i)/(nnz/p)$ tells us the deviation from this ideal case, where a ratio of one is a perfect balance and larger values indicate growing inefficiency. Banded matrices

Table 3.4: Memory consumption at selected sparsity levels for a 10000×10000 matrix.

Sparsity (%)	nnz	Dense	COO	CSR/CSC	Winner
0.0	100 000 000	381.47 MB	1.118 GB	762.98 MB	Dense
50.0	50 000 000	381.47 MB	572.20 MB	381.51 MB	Dense
75.0	25 000 000	381.47 MB	286.10 MB	190.77 MB	CSR/CSC
90.0	10 000 000	381.47 MB	114.44 MB	76.33 MB	CSR/CSC
95.0	5 000 000	381.47 MB	57.22 MB	38.19 MB	CSR/CSC
99.0	1 000 000	381.47 MB	11.44 MB	7.67 MB	CSR/CSC

Table 3.5: Sparsity crossover points at which one format becomes more memory efficient than another, for three matrix sizes.

Transition	$N = 100$	$N = 1000$	$N = 10000$
COO beats Dense	$\sim 66.7\%$	$\sim 66.7\%$	$\sim 66.7\%$
CSR/CSC beats Dense	$\sim 50.5\%$	$\sim 50.1\%$	$\sim 50.0\%$
CSR/CSC beats COO	below $\sim 99.0\%$	below $\sim 99.9\%$	None (CSR always wins)

often produce high imbalance when only processes covering the diagonal rows hold the non zeroes. Block matrices hold a similar pattern at the block level. For Kronecker matrices, they are expected to produce the most severe imbalance as their power law degree structure concentrates non zeroes into a small fraction of rows that grows with the matrix size.

Load imbalance also compounds cost with the metadata overhead discussed in Section 3.1.2. A heavily loaded process must construct and transmit a proportionally large index array, while a lightly loaded process still pays the full startup latency α for a near empty message. Neither end of the imbalance cost benefits from additional parallelism, which is a core reason why sparse collectives exhibit worse strong scaling than the dense counterparts.

3.4 Expected Effects by Sparsity Pattern

The four sparsity patterns studied in this thesis interact with the collective cost in distinct ways, based on the previous analysis we performed.

For uniform random sparsity, each process receives approximately nnz/p non zeroes regardless of the partition boundaries, so message sizes are nearly equal across processes and steps. Collective cost here should scale linearly with density, and thus makes uniform sparsity the most predictable and useful as a baseline for the other patterns.

For banded sparsity, cost depends entirely on how the band aligns with the process partition boundaries. When the band is narrow and processes are assigned row ranges, only a small number of processes hold a significant number of non zeroes which will create a high degree of imbalance in scatter and gather operations. As bandwidth grows toward the full matrix, load imbalance improves and the resulting cost will grow closer to that of a uniform sparsity matrix. Broadcast should be less sensitive to this as all processes will achieve the same message.

For block sparsity, imbalance depends on whether block positions are spread evenly across the process grid and how blocks are distributed. Well distributed blocks should produce predictable message sizes and thus good overall balance. Clustered blocks will concentrate the load on a small number of processes. The best case here is that block sparsity is the most communication efficient pattern because it can eliminate the message size overhead if blocks align with process partitions.

For Kronecker sparsity, we expect that it will produce the worst case scenarios for every collective and operation. Because the hierarchical degree structure guarantees that a small fraction of rows account for a large share of the non zeroes, collectives like scatter and gather will be cost dominated by those processes. As the matrix size

and thus number of non zeroes increase (a result of increased Kronecker iterations), the imbalance ratio and total cost will worsen dramatically.

3.5 Scaling Behavior

3.5.1 Strong Scaling

Strong scaling refers to the behavior of an algorithm when the problem size is held fixed and the number of processes is increased. In the ideal case, doubling the number of processes halves the time for a fixed workload, essentially achieving a linear speedup. For dense collective communications, there is an understood strong scaling behavior: the bandwidth cost decreases with additional processes while the latency cost increases due to growing communication rounds.

For sparse collectives, strong scaling is of course subject to a new constraint: as the number of processes increase, the average number of non zero entries per process decreases as $\frac{nnz}{p}$. Once $\frac{nnz}{p}$ drops below some threshold at which communication overhead dominates computation, further scaling will yield diminishing returns. This threshold arrives much earlier for sparse data than dense data because the effective message size decreases faster and the metadata overhead becomes a large portion of the message.

3.5.2 Weak Scaling

Weak scaling refers to the behavior of an algorithm when the problem size is scaled proportionally with the number of processes, such that each process will hold a fixed amount of work. In the ideal case, total time remains constant as both the problem size and process count increase together. For dense collectives, weak scaling is limited

by the latency cost which grows even when the total data volume is fixed because more processes must be coordinated.

For sparse collectives, weak scaling has an additional complexity - if the problem is scaled by adding more rows or columns while keeping the sparsity and structure fixed, the per process non zero count remains approximately constant. However, the metadata overhead also grows with the number of rows per process in CSR and CSC formats, even if the non zero count is constant, because the row pointer array scales with the number of rows. For matrices where the number of rows per process is large relative to the average number of non zeros per row, this can cause weak scaling to be significantly worse than the dense case.

3.6 Summary

This chapter has analyzed how the seven collective communication operations introduced in Chapter 2 are affected when operating on sparse data. The core difficulty here is that sparse formats violate all of the fixed message size assumptions that collective algorithms are often designed around.

Chapter 4

Sparse SUMMA

This chapter extends the analysis of the SUMMA algorithm introduced in Chapter 2 to the case where one or both input matrices are sparse. The transition from dense to sparse SUMMA requires revisiting nearly every phase of the algorithm: the data distribution, the broadcast operations, and the local multiplication step. Each of these phases introduces new complexity when data is sparse and the appropriate choices for storage format, partitioning strategy, and communication primitives depend on the properties of the input matrices. The goal of this chapter is to develop a theoretical understanding of how sparsity interacts with the SUMMA structure and communication which will inform further experiments.

4.1 From Dense to Sparse SUMMA

4.1.1 Overview

Recall from Chapter 2 that the dense SUMMA algorithm computes $C = A \times B$ by distributing all three matrices across a $P_r \times P_c$ process grid and iterating $\frac{k}{b}$ panel steps.

At each step s , the process in column s broadcasts its local A panel across its process row, and the process in row s broadcasts its local panel B down its process column. Every process then computes the local outer product contribution and accumulates it into its local block of C.

In the dense case, the A panel broadcast transmits a contiguous block of m/P_r rows, each of length k/P_c , for a total message of size $(m/P_r) \times (k/P_c)$ elements. The B panel broadcast transmits a contiguous block of k/P_r rows, each of length n/P_c , for a total message size of $(k/P_r) \times (n/P_c)$ elements. Both message sizes are deterministic and equal across all steps and all processes.

When A and B are sparse, neither of those properties hold true. The A panel for step s is the submatrix of A consisting of the rows held by the broadcasting process and the columns corresponding to the s -th panel. The submatrix may be empty, partially full, or nearly dense depending on where the non zero entries of A are located relative to the panel boundaries. Similarly for the B panel. The result is that message sizes vary across steps and across processes in a pattern determined entirely by the sparsity structure of the input matrices.

4.1.2 Sparsity Challenges

The variability in message sizes creates several practical challenges for the sparse SUMMA implementation. First, the receiving processes cannot preallocate buffers of the correct size without first knowing how many nonzero entries it will receive. This requires as previously mentioned an additional metadata exchange before each broadcast, add least adding one additional latency term per step. Second, some panels may be entirely empty, in which case the broadcast carries no data but still incurs some message cost as the receiving processes do not know they will receive no data.

Third, the local multiplication that follows each broadcast is a sparse times sparse (SpGEMM) operation rather than a simple dense matrix multiply. SpGEMM is substantially more complex than dense multiplication: the output sparsity pattern is not known in advance and the computation is likely irregular in access patterns. Although not the focus of this thesis as there are many works on SpGEMM and general sparse times sparse multiplication [15], it still does present a challenge to note.

4.2 Storage Format Implications

4.2.1 CSR and A-Panel Broadcasts

The A panel broadcast in SUMMA distributes rows of A across the process row communicator. As discussed in Chapter 2, the CSR format organizes nonzero entries row by row, making it a natural choice for row oriented operations. For the A panel broadcast, the broadcasting process must extract the rows of its local A block that belong to the current panel step and pack them into a message. In CSR, this extraction is a simple slice of the values and col_indices array between row_ptrs[i] and row_ptrs[i + 1] for each relevant row i . Thus, this is a $O(nnz_{panel})$ operation where nnz_{panel} is the number of nonzeros in the panel as it does not require a scan of the entire local matrix.

4.2.2 CSC and B-Panel Broadcasts

The B panel broadcast in SUMMA distributes columns of B across the process column communicator. The CSC format organizes non zero entries column by column, making it the natural format for these column orientated operations. For the B panel

broadcast, the broadcasting process must extract the columns of its local B corresponding to the current panel step. In CSC, this is again a simple slice between `col_ptrs[j]` and `col_ptrs[j + 1]` for each relevant column j , which is $O(nnz_{panel})$.

4.2.3 Format Tradeoffs

The choice of storage format for either of the inputs as previously noted has a direct impact on both the efficiency of the broadcast and the local multiply phases. Using CSR for both A and B supports efficient A panel extraction and is compatible with standard row wise SpGEMM kernels, but makes B panel extraction inefficient because accessing a column of a CSR matrix requires scanning all rows. Using CSC for both supports efficient B panel extraction but makes A panel extraction inefficient for the same reason. A mixed strategy, storing A in CSR and B in CSC supports efficient extraction for both panels but requires the format to be specified and maintained separately. For later experiments and evaluation, both CSR/CSC only and CSR/CSC strategies are implemented and compared.

4.3 Matrix Shape and Process Grids

4.3.1 Square vs Rectangular Matrices

The SUMMA algorithm is inherently designed for arbitrary matrix shapes, but the choice of process grid dimensions P_r and P_c interacts with matrix shape in ways that affect both communication cost and load balance. For a square matrix of dimension $n \times n$ distributed over $P_r \times P_c$ grid, each process holds approximately n/P_r rows and n/P_c columns. The A panel broadcast sends messages of size proportional to $(n/P_r) \times (k/P_c)$, and the B panel sends messages of size proportional to $(k/P_r) \times (n/P_c)$. For

a square process grid where $P_r = P_c = \text{sqrt}(P)$, these two message sizes are equal and the broadcast load is symmetric across row and column communicators.

For rectangular matrices, or for non square grids, the two broadcast message sizes are no longer equal. A tall and skinny matrix for example benefits from a process grid with many rows and few columns, which reduces the per process row count and distributes the A panel broadcasts efficiently. A wide and short matrix benefits from the opposite configuration. For sparse matrices, this interaction is further complicated by the fact that the actual message sizes depend on the non zero distribution, not just the matrix dimensions. A tall matrix with banded sparsity may have very few non zeroes per row in the off diagonal region of each panel, making the A panel messages small regardless of process shape.

4.4 Sparsity Pattern Effects

4.4.1 Communication Bottlenecks vs Local Multiplication

In dense SUMMA, the relative cost of communication and local computation is bound by the ratio at which the local dense multiply can process data, which is well studied and typically favors computation at scale. For sparse SUMMA this balance shifts because the local SpGEMM performs fewer arithmetic operations but generates irregular memory access patterns that are less amenable to hardware optimization. The result is that instead, communication and computation are more closely matched in cost for sparse SUMMA, and in fact the dominant bottleneck will likely be communication based on the sparsity pattern and sparsity of the matrix.

For highly sparse matrices, the local SpGEMM in each step is fast because the A and B panels contain few non zeroes. The per step communication cost, however, still

incurs a startup latency of at least α per broadcast even when panels are empty. As sparsity increases, the computation cost drops proportionally to nnz , but latency cost remains fixed. The result here is that for very low densities or high sparsity, sparse SUMMA becomes communication latency bound rather than computation bound, and scaling to more processes makes the problem worse by increasing the number of broadcasts without reducing the startup latency.

4.4.2 Expected Behaviors by Pattern

For uniform sparsity, the A and B panels at each step statistically contain approximately the same number of non zeroes. Communication is therefore balanced across steps and processes, and sparse SUMMA cost is approximately the dense SUMMA cost scaled by the sparsity any additional communication overhead incurred by the additional metadata depending on matrix size.

For banded sparsity, the panel structure interacts strongly with the banded structure. Steps who panel range falls within the band produce large, dense panel messages, while steps outside the band produce empty or near empty panels. The total communication volume is therefore concentrated into a small number of steps, which may or may not benefit performance depending on whether those steps can be pipelined. The expected behavior is that the banded sparse SUMMA is significantly faster than uniform sparse SUMMA at the same sparsity, because most steps contribute no communication or computation.

Missing here is a section on both block and kronecker patterns for sparse SUMMA.

4.5 Summary

Chapter 5

Methodology and Experimental Setup

This chapter describes the experimental methodology used to evaluate the theoretical predictions developed in Chapters 3 and 4. The experimental design is organized around five of the previously mentioned variables: matrix shape and sparsity level, sparsity pattern, storage format, and parallelism/scaling. Each experiment holds a subset of these variables fixed while varying others, allowing us to attribute the performance differences to specific factors. All experiments are repeated multiple times to account for variability, and all statistics are reported over these repetitions.

5.1 Matrix Shape and Sparsity

Square matrices with dimensions $n \times n$ will be used as the primary test case, with n ranging in size based on the experiment. Non square matrices are also used to study the effects of a matrices aspect ratio on SUMMA and collective performance, with aspect ratios similarly ranging based on the experiment. Aspect ratios are designed

to test matrices that are "tall and skinny" or "wide and short". For each of these variations, sparsity levels that range from as low as 0.0001% sparsity to a standard dense matrix will also be tested.

Synthetic matrices are generated for each combination of shape, density, and sparsity pattern. For uniform random sparsity, entries are sampled uniformly at random. For banded sparsity, all entries within a specified bandwidth of the main diagonal are populated with uniform random values. For block sparsity, a set of block positions is randomly selected and each block is filled densely.

Kronecker matrices were generated by Moez and need to be cited as such here. I'd like to explain what datasets we are using from Moez, the range of values, how he generated them (citations to the kronecker paper are here somewhere), etc.

5.2 Parallelism

All distributed experiments are run on process counts varying counts, typically in even increments (e.g. 2, 4, 6) and in square increments (e.g. 3, 9, 16). On top of process counts varying, the amount of nodes requested on the super computer may vary as well and will be listed as such in any experiments.

Strong scaling experiments fix the total matrix size and vary the process count. Weak scaling experiments increase the matrix size proportionally with the process count such that each process holds approximately the same number of non zero entries.

5.3 Hardware and Software

All experiments that are used for statistics (e.g. tables, charts, etc) are conducted on the Oklahoma Supercomputing Center for Educating and Research (OSCER) cluster at the University of Oklahoma [16].

List a table of some of the basic specs of the super computer nodes.

The implementation is written in C++ and compiled with MpiCC using standard optimization flags. The following table describes specific versions used for Open-MPI, MPI, and any other software. Sparse matrix storage and generation utilities are implemented from scratch without dependence on external sparse linear algebra libraries, in order to maintain full control over the communication patterns and conversions. However, other implementations of sparse multiplication libraries such as CombBLAS may be used as baselines and comparisons [17, 18].

5.4 Metrics and Instrumentation

5.4.1 Custom Profiling Tool

In order to facilitate the testing required for future experiments, a custom profiling tool was built that can measure a variety of variables and automatically generate per rank graphs and tables based on the outputs. Because many of the algorithms implemented in this thesis are complex and involve multiple levels of communication, the ability to profile sub scopes of the code was required. Listing 6 demonstrates an example of the SUMMA algorithm scoped by individual phases, specifically: a k iteration of the algorithm, the a and b panel broadcasts, and the local computation.

This allows us to create statistics that represent the algorithm as a whole, but also view data like communication vs computation comparisons.

```

1  MatA summa(const MatA& a_local, const MatB& b_local, const SummaGrid& g)
2  {
3      ...
4      for (int k = 0; k < g.grid_rows; ++k)
5      {
6          perf::Scope summa_scope("summa_iteration" + std::to_string(k));
7          MatA a_panel = (g.col_coord == k) ? a_local : MatA{};
8          {
9              perf::Scope scope("summa_apanel_bcast");
10             summa_bcast(a_panel, k, g.row_comm);
11         }
12
13         MatB b_panel = (g.row_coord == k) ? b_local : MatB{};
14         {
15             perf::Scope scope("summa_bpanel_bcast");
16             summa_bcast(b_panel, k, g.col_comm);
17         }
18
19         {
20             perf::Scope scope("summa_computation");
21             summa_accumulate(c_local, a_panel, b_panel);
22         }
23     }
24     ...
25 }

```

Listing 6: A snippet of the sparse SUMMA implementation with profiling for individual scopes.

5.4.1.1 PMPI

PMPI is a small library used to intercept OpenMPI calls used for creating more in-depth profiling tools that can individually measure each call [19]. Our custom profiling tool uses this to fully breakdown functions not only by the custom wrappers, but also by MPI calls. Whenever a stacks runtime is calculated, the total MPI runtime within that stack is also calculated so that the remainder of the code within a stack can be determined. An example of this behavior is shown below.

5.4.1.2 Output

The output of the profiling tool is a folder that is uniquely named by the current timestamp at the time of initialization. All ranks are given this time via an OpenMPI call and thus all log to the same directory. The directory itself is then split into a few different files that all contain varying data.

1. /ranks/x.json - Contains information about that ranks individual profiling such as MPI call statistics, stack information, etc.
2. /metadata.json - Contains information about the current state of the project whenever the profiling is initialized such as the current git commit/branch, arguments, etc.
3. /system.json - Contains information about the system the test was ran on such as total memory, the CPU, the GPU, etc. In the event that more than 1 node was used (as determined by how the experiment was created), this information may contain an array of systems.

While the information in metadata.json and system.json are not particularly interesting, containing just generic system and project level metadata as mentioned.

However, the individual ranks data contain nested data relating to the overall processes. For each scope, information including the total execution time, any children (scopes executed within a scope), mpi calls and mpi bytes, etc. are recorded. Listing 7 contains an example of a ranks JSON from a small MST Scatter experiment.

```
1  {
2      "name": "experiment_iteration",
3      "wall_time_s": "123.333",
4      "values": {
5          "rank": 2,
6          "world_size": 8,
7          "iteration": 0,
8          "experiment": "mst_scatter_banded"
9      },
10     "children": [
11         {
12             "name": "mst_scatter",
13             "mpi_calls": {
14                 "MPI_Barrier": 990,
15                 "MPI_Recv": 3960
16             },
17             "mpi_bytes": {
18                 "MPI_Barrier": 0,
19                 "MPI_Recv": 3189019200
20             }
21         }
22     ]
23 }
```

Listing 7: A portion of a ranks output data from an experiment.

Chapter 6

Results and Analysis

This chapter presents and interprets the experimental results collected using the methodology described in Chapter 5. The results are organized into four sections. Section 6.1 discusses the experimental design choices and the characterization of the variability in the measurements. Section 6.2 establishes baselines for both the collective communication and SUMMA experiments. Section 6.3 and 6.4 present results for sparse collective communication and sparse SUMMA respectively, organized by the primary variables: sparsity level, sparsity pattern, storage format, and process count.

6.1 Experiment Design and Variability

As described in Chapter 5, each experimental configuration is run multiple times and the median results are used as the primary performance statistic. The choice of median over mean is chosen to filter out occasional outliers caused by operating system jitter, network contention, or memory allocation delays that inflate the mean without reflecting the real behavior of the experiments.

6.2 Baselines

There are a few core baselines that will go here soon. Most importantly we will do a small dense analysis of the collectives (not too in depth, since the collectives paper already did this), and a analysis of the CombBLAS performance. While we certainly do not expect to beat the CombBLAS performance in any meaningful way, we can use it as a "here is the upper bound for performance, we represent the lower bound given these collectives, sparsity patterns, etc".

6.3 Collective Communication Results

This section will focus on running all of the collective communications discussed in Chapter 2 and analyzing the effects of sparse matrices. Because the introduction of sparsity introduces so many different knobs we can adjust, we will take a look at all of them individually and then view the effects when combined.

6.3.1 Impact of Sparsity Level

6.3.1.1 MST Collectives

This section will analyze the impact of matrix sparsity on all of the MST collectives with a focus on a fixed matrix size of $n = m = 4096$ and $p = 8$ where p is the number of processes. Figure 6.1 has all four MST collectives in one visualization. This section will continue to analyze each individually.

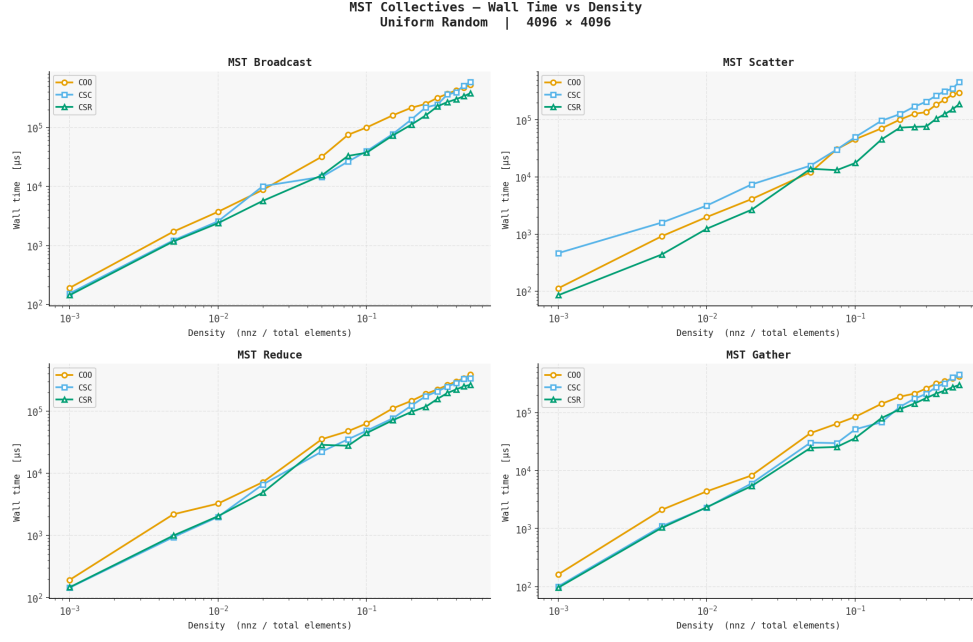


Figure 6.1: Sparse MST Collectives wall time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

Figure 6.2 shows the broadcast time as a function of matrix sparsity for a uniform random sparsity pattern with a fixed matrix dimension and 8 processes. As expected based on our analysis in Chapter 2, COO is the slowest out of the three formats in terms of wall time whilst CSR and CSC are nearly identical in performance as density increases.

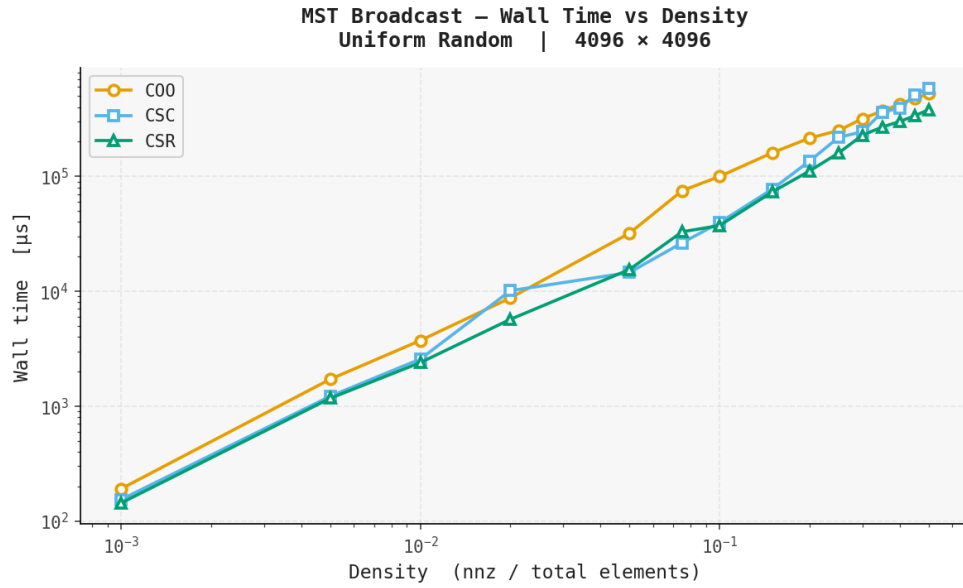


Figure 6.2: Sparse MST Broadcast time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

Figure 6.3 shows the gather time as a function of matrix sparsity for a uniform random sparsity pattern with a fixed matrix dimension and 8 processes. Similar to the MST Broadcast collective, COO is the slowest whilst CSC and CSR are nearly identical in performance as density increases.

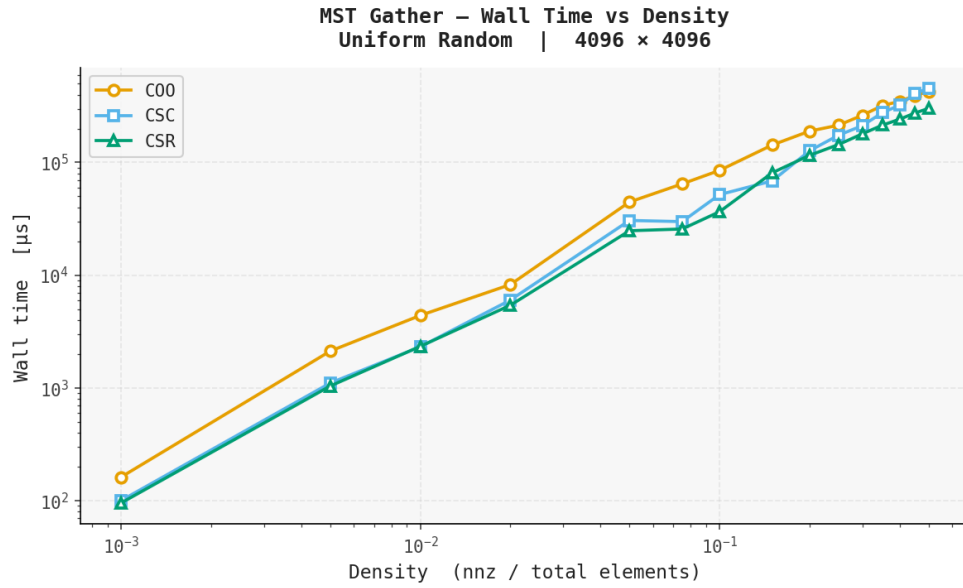


Figure 6.3: Sparse MST Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

Figure 6.4 shows the reduce time as a function of matrix sparsity for a uniform random sparsity pattern with a fixed matrix dimension and 8 processes. Again, this shows nearly identical behavior to both MST Broadcast and MST Gather.

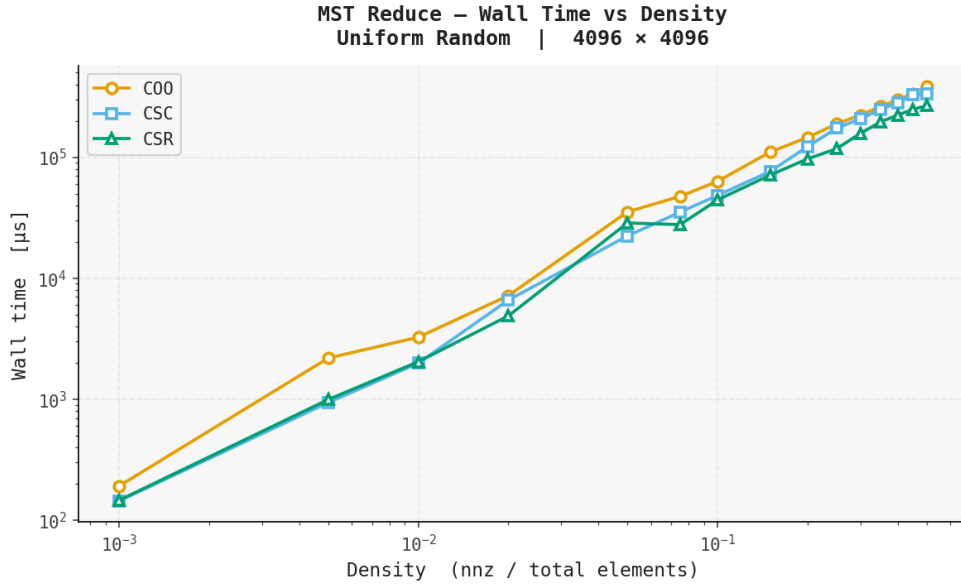


Figure 6.4: Sparse MST Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

Figure 6.5 shows the scatter time as a function of matrix sparsity for a uniform random sparsity pattern with a fixed matrix dimension and 8 processes. Unlike the broadcast, gather, and reduce collectives the scatter operation reveals a notable change between CSC and the other two formats. CSC is consistently the slowest across the full density range, whilst CSR is the fastest and COO is between. This arises from the directional nature of the scatter we implemented, where the root process partitions the matrix in matrices by row. This appears to be the most common implementation of scatter, especially for algorithms like SUMMA. COO avoids having to do any reorganization like CSC, but still is costly due to the larger metadata footprint it carries.

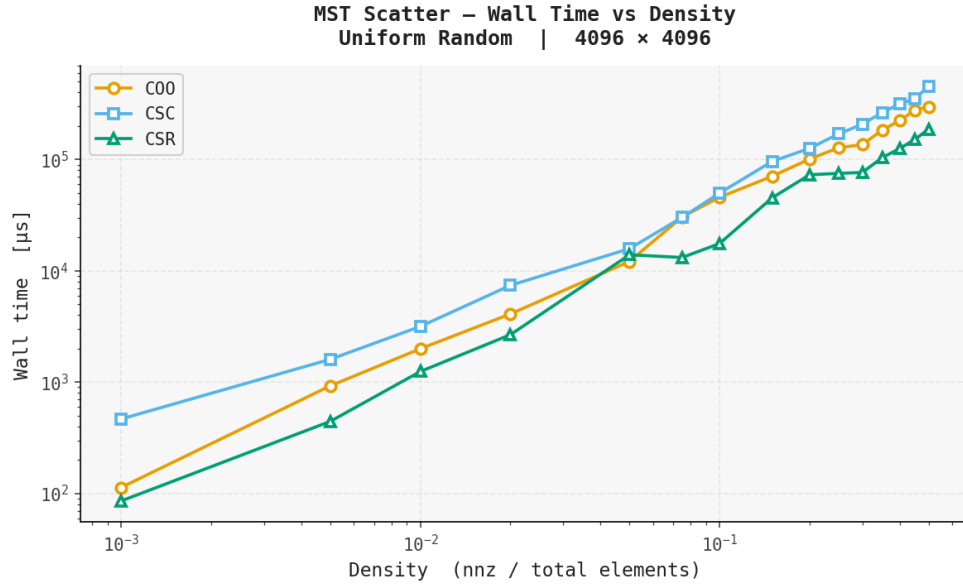


Figure 6.5: Sparse MST Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

6.3.1.2 Bucket Collectives

This section will analyze the impact of matrix sparsity on all of the Bucket collectives with the same matrix configuration as described in the MST collectives section. Figure 6.6 has all three Bucket collectives in one visualization. This section will continue to analyze each individually.

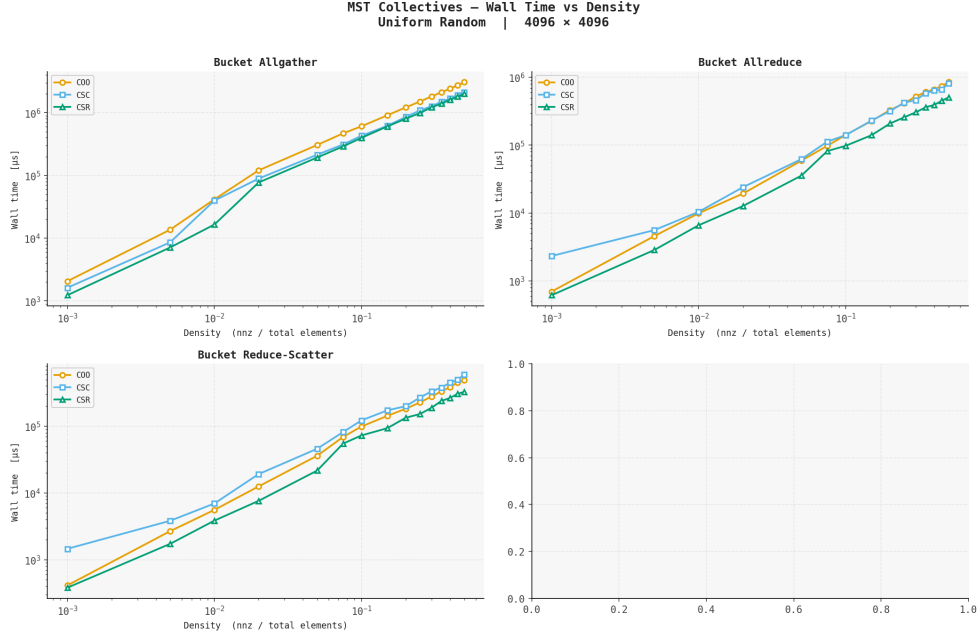


Figure 6.6: Sparse Bucket Collectives wall time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

Figure 6.7 shows the bucket all gather time as a function of matrix sparsity for a uniform random sparsity pattern with a fixed matrix dimension and 8 processes. The results here are quite similar to the MST counterpart, however, one important statistic to note is that the wall times are substantially larger. This is of course a direct result of all gathers communication, where every process must receive data from all other processes. As expected, CSR and CSC are very similar in performance while COO is slow overall due to the metadata overhead.

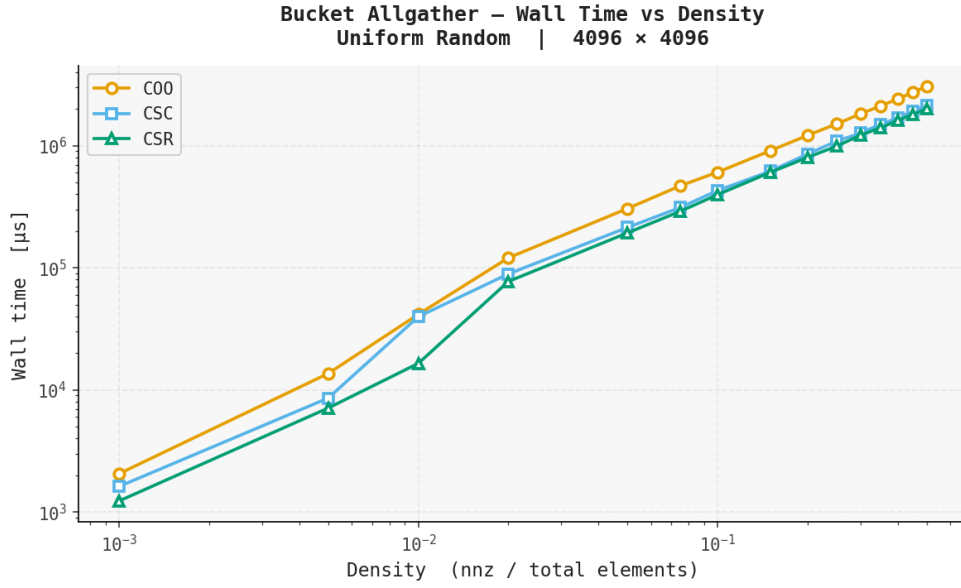


Figure 6.7: Sparse Bucket All Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

Figure 6.8 shows the bucket all reduce time as a function of matrix sparsity for a uniform random sparsity pattern with a fixed matrix dimension and 8 processes.

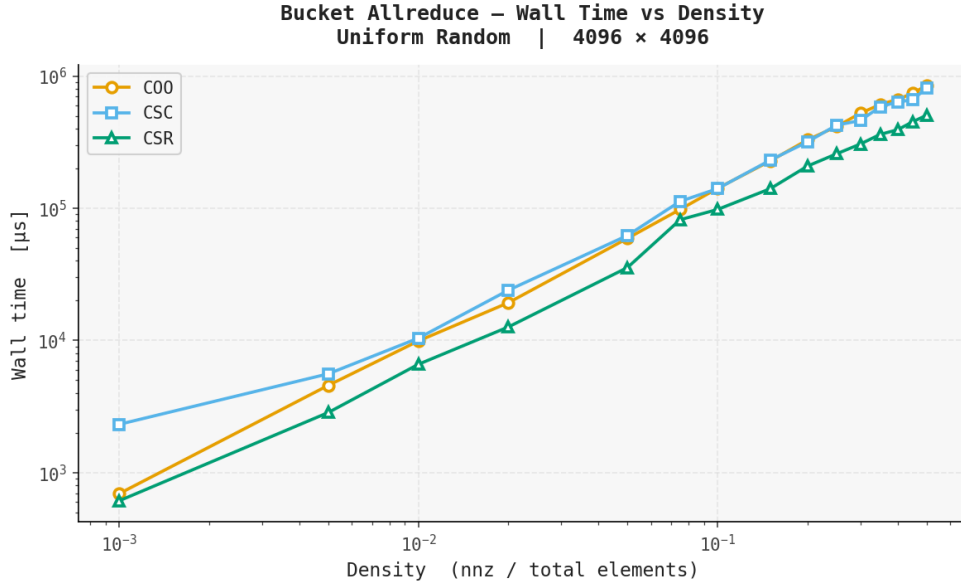


Figure 6.8: Sparse Bucket All Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

Figure 6.9 shows the bucket reduce scatter time as a function of matrix sparsity for a uniform random sparsity pattern with a fixed matrix dimension and 8 processes. CSC appears to be the slowest format across the range, whilst CSR is the fastest and COO is the slowest. This of course mirrors the behavior we observed in the MST Scatter operation, in which the CSC format must perform additional work because the data is not indexed in a form that is most efficient for CSC.

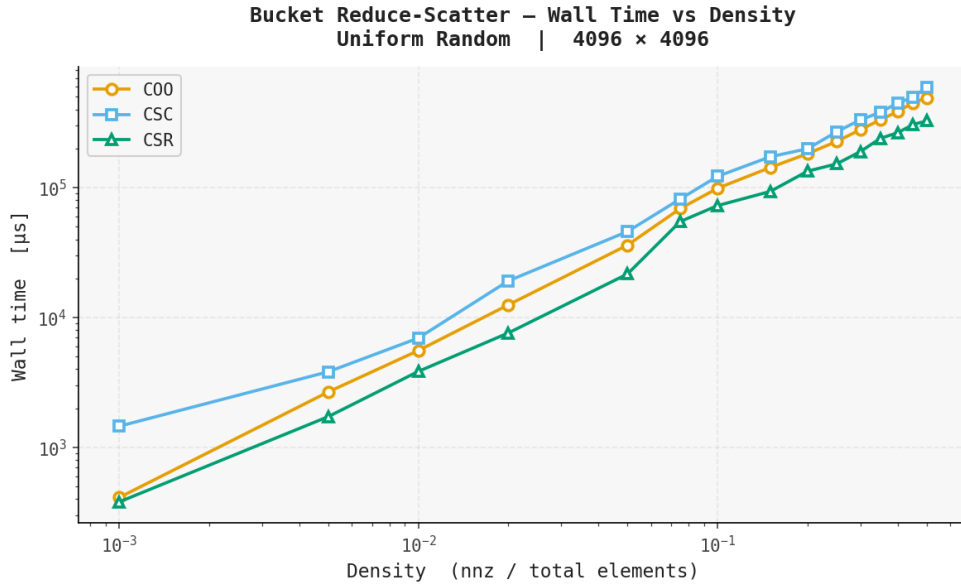


Figure 6.9: Sparse Bucket Reduce Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

A similar behavior can be observed for all of the above collectives when changing the dimensions between both square matrices and rectangular matrices (see Appendix ??).

6.3.2 Impact of Sparsity Pattern

This section focuses on the Sparsity Patterns we have implemented in this thesis such as blocked or banded and how they effect the performance of our collectives. Each

experiment was ran using a similar configuration as described before.

6.3.2.1 MST Collectives

Figures 6.10 through 6.13 visualize wall time versus three different sparsity patterns and storage formats at a fixed density, matrix size, and number of processes. The most obvious result here for all of the figure is that the uniform random pattern clearly under performs compared to the rest of the results. This is expected based on all of our previous analysis, as even at a 5% density almost every process evenly holds a roughly equal share of the data.

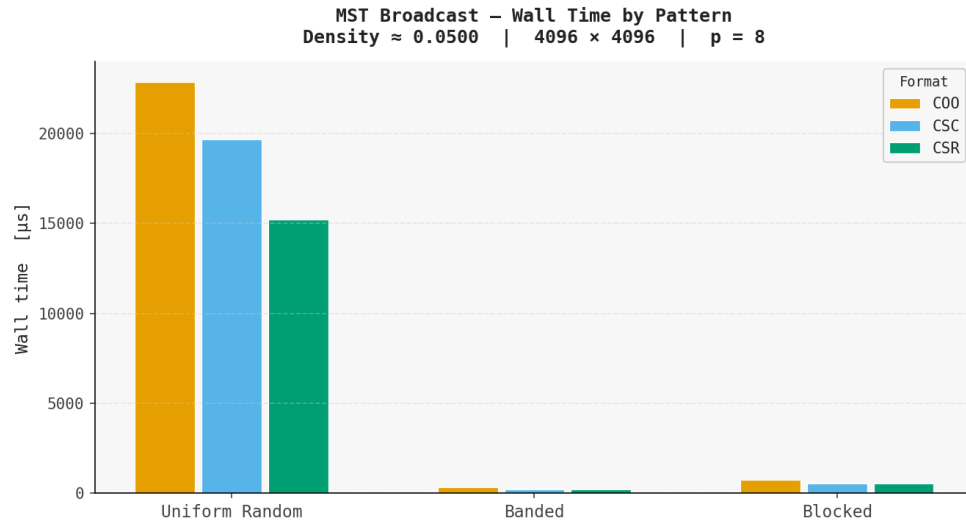


Figure 6.10: Sparse MST BCast time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

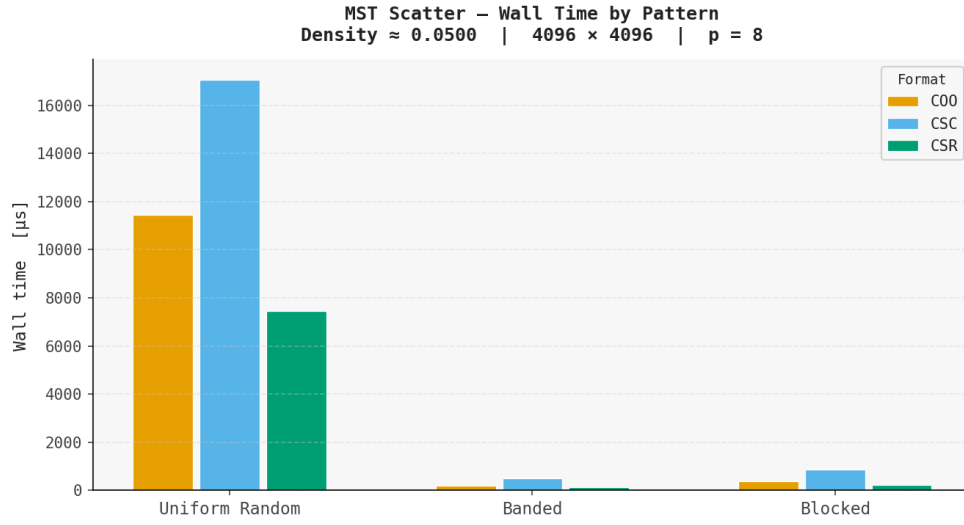


Figure 6.11: Sparse MST Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

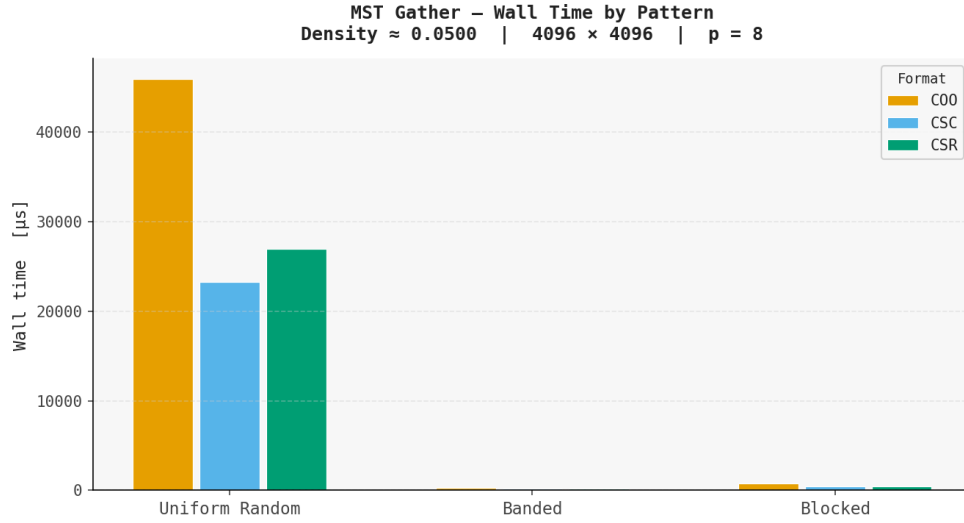


Figure 6.12: Sparse MST Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

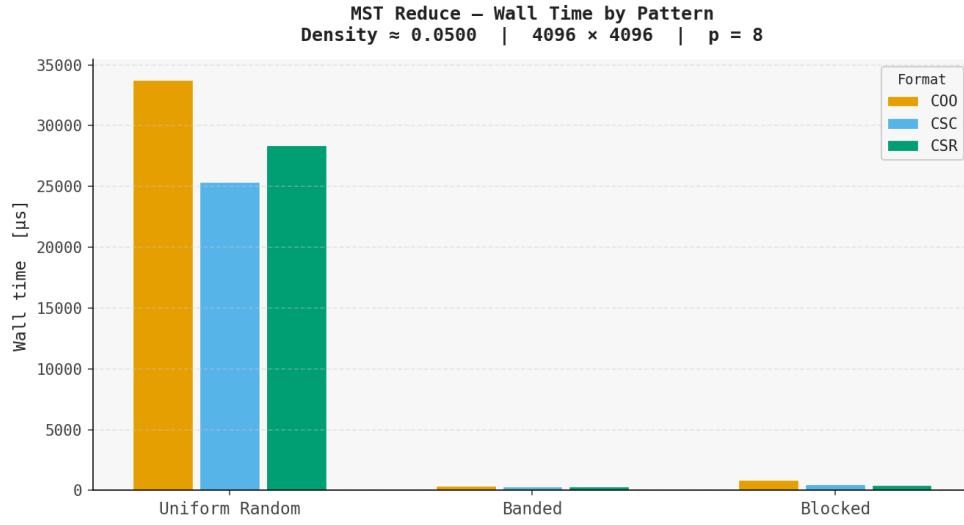


Figure 6.13: Sparse MST Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

6.3.2.2 Bucket Collectives

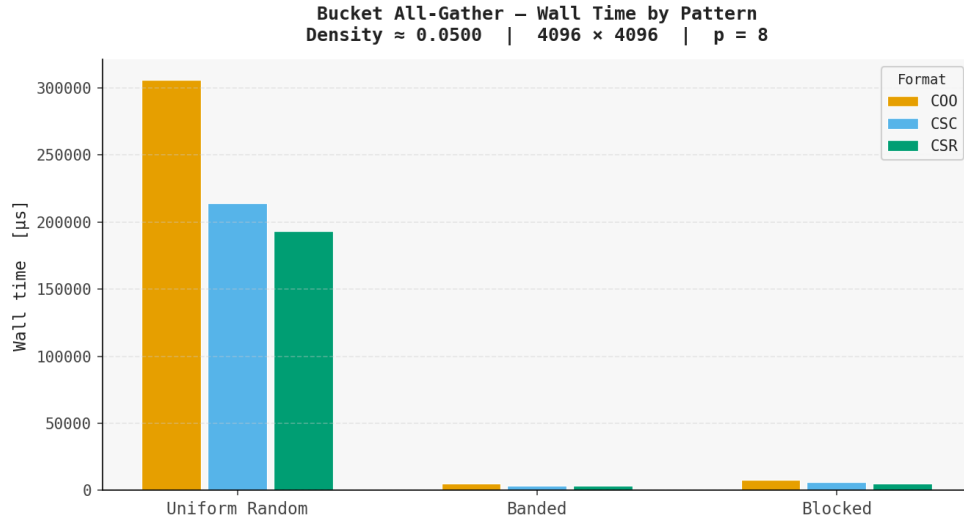


Figure 6.14: Sparse Bucket All Gather time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

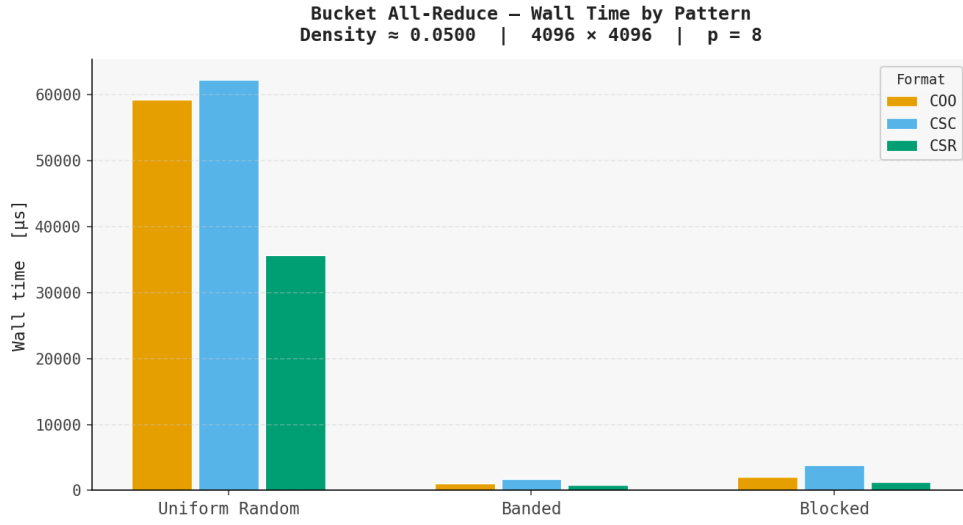


Figure 6.15: Sparse Bucket All Reduce time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

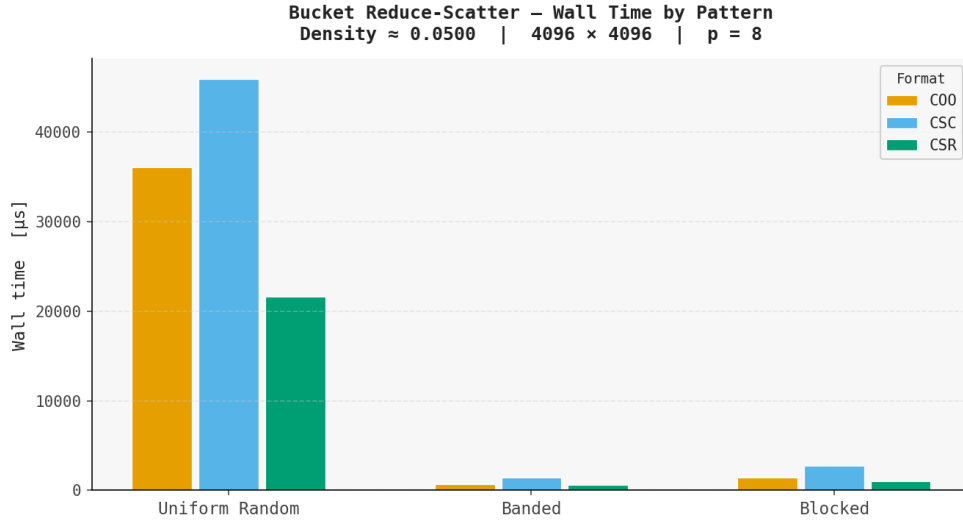


Figure 6.16: Sparse Bucket Reduce Scatter time as a function of matrix sparsity for a uniform random sparsity pattern ($n = 4096$, $p = 8$)

6.3.3 Scaling Behavior

This section focuses on understanding how the sparse collectives behave as the number of participating processes increases. As discussed in Chapter 2, the expected

behavior of scaling processes is that at a certain point the weight of communication will outweigh the computation and thus will become more inefficient.

Demonstrate how these collectives scale (with both sparsity pattern and level) along with the number of processes and nodes running alongside the program.

6.3.4 Combined Effects

Show a few examples of all of the above parameters being changed together, and how that effects results. Although this should be fairly obvious by simply combining the above graphs mentally, it will be nice to show it visually and statistically.

6.4 SUMMA Results

6.4.1 Impact of Sparsity Level

Figure 6.17 shows sparse SUMMA wall time as a function of matrix density across the three different sparsity patterns, using CSR for both inputs on a 4096×4096 matrix with 8 processes. The results do confirm the prediction we made back in Chapter 4, where wall time scales roughly with density. However, the relationship as seen in the graph is not perfectly linear due to the per step latency cost regardless of the number of non zeroes.

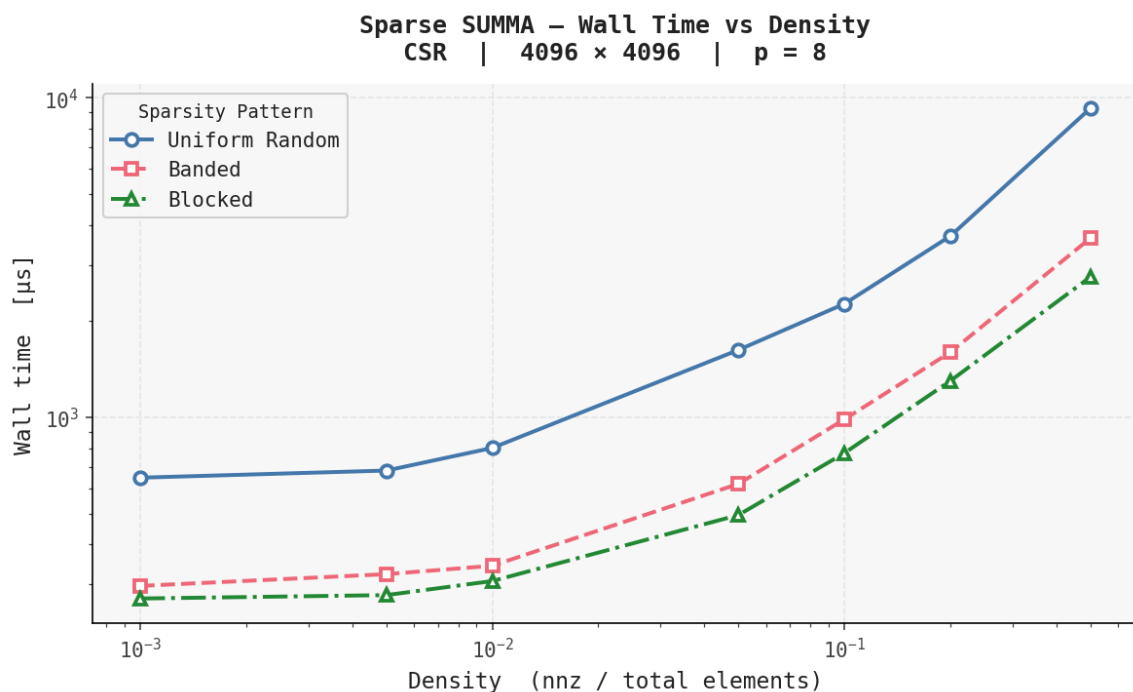


Figure 6.17

Uniform random sparsity consistently produces the highest wall times at any given density level. This is of course expected because the panels at every step of the algorithm carry non zero data proportional to the overall density, meaning that all k steps contribute both meaningful communication and computation. Banded and blocked sparsity patterns perform much better at equivalent densities because their non zeros are concentrated, leaving many panels empty or near empty. However, as the density increases all three patterns begin to converge since the panels become dense enough that structured patterns begin to lose their advantage. At very low densities, the curve flattens because of the communication latency discussed earlier. Because even when the panels are nearly empty, each broadcast step still incurs some startup cost α so lower densities do not yield better results.

6.4.2 Impact of Sparsity Pattern

Figure 6.18 compares sparse SUMMA wall time across three sparsity patterns at a fixed density of around 0.05, using all of the available format strategies: CSR only, CSC only, and a mixed CSR + CSC approach.

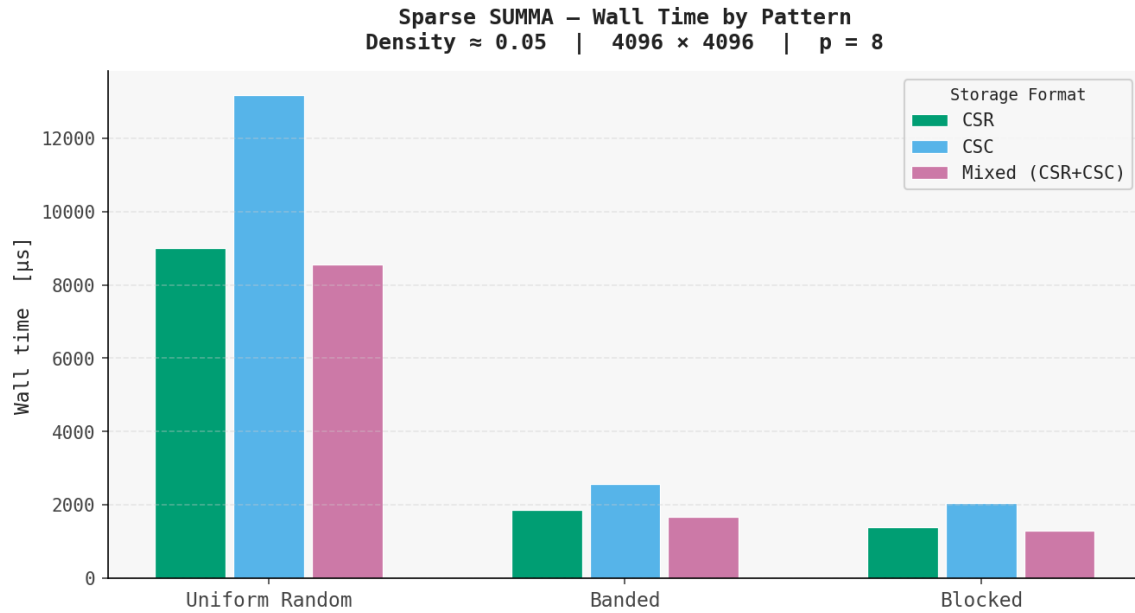


Figure 6.18

The most interesting result here is that there is quite a large gap between uniform random and the structured pattern, although perhaps not all that surprising. At this density, both banded and blocked matrices complete in roughly a fifth of the time required as by the uniform random case. This matches the prediction we made earlier, that structured patterns concentrate non zeroes into a subset of panel steps, leaving the remaining steps with low communication and computation cost. The uniform random pattern distributes non zeroes evenly, so no step is cheap.

6.4.3 Impact of Matrix Dimensions

Figure 6.19 shows sparse SUMMA wall time as a function of total matrix elements for three different matrix shapes: square, tall and skinny, and wide and short. All of these are using the CSR only format at around 5% density and 8 processes.

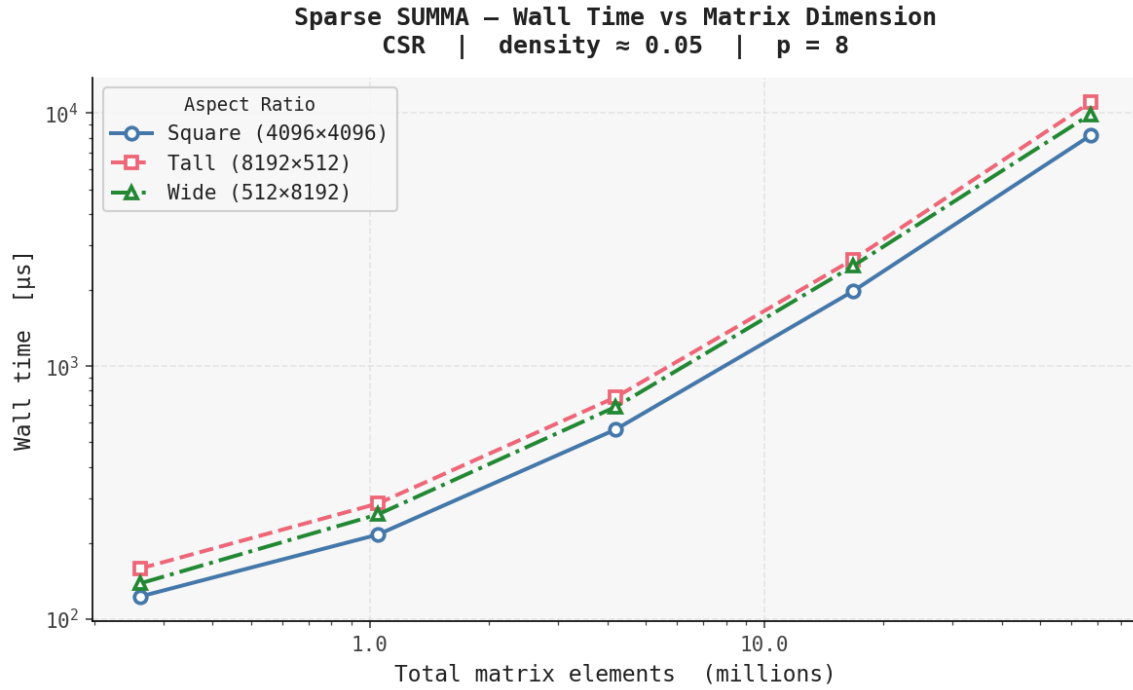


Figure 6.19

All three of these configurations scale similarly in total wall time as the number of elements increases. It should however be noted that the tall and wide configurations consistently run slightly slower than the square configuration at equivalent element counts. This can best be explained by our earlier discussion on how the process grid interacts with the matrix shape.

As the problem size grows it appears that the three shapes converge in their scaling, suggesting that the per element cost of sparse SUMMA is approximately agnostic at this configuration. The offset between square and non square configurations is cer-

tainly because of the process grid alignment, such that square matrices map evenly to square process grids, and wide or tall matrices produce unequal panel sizes.

6.4.4 Impact of Storage Format

Figure 6.20 shows parse SUMMA wall time as a function of density for a uniform random matrix, with similar configuration to before. At low densities, it is clear that CSR is the fastest single format option which is consistent with how we expected it to perform. A mixed strategy when aligned properly, which uses CSR for the A matrix and CSC for the B matrix, is by far the fastest.

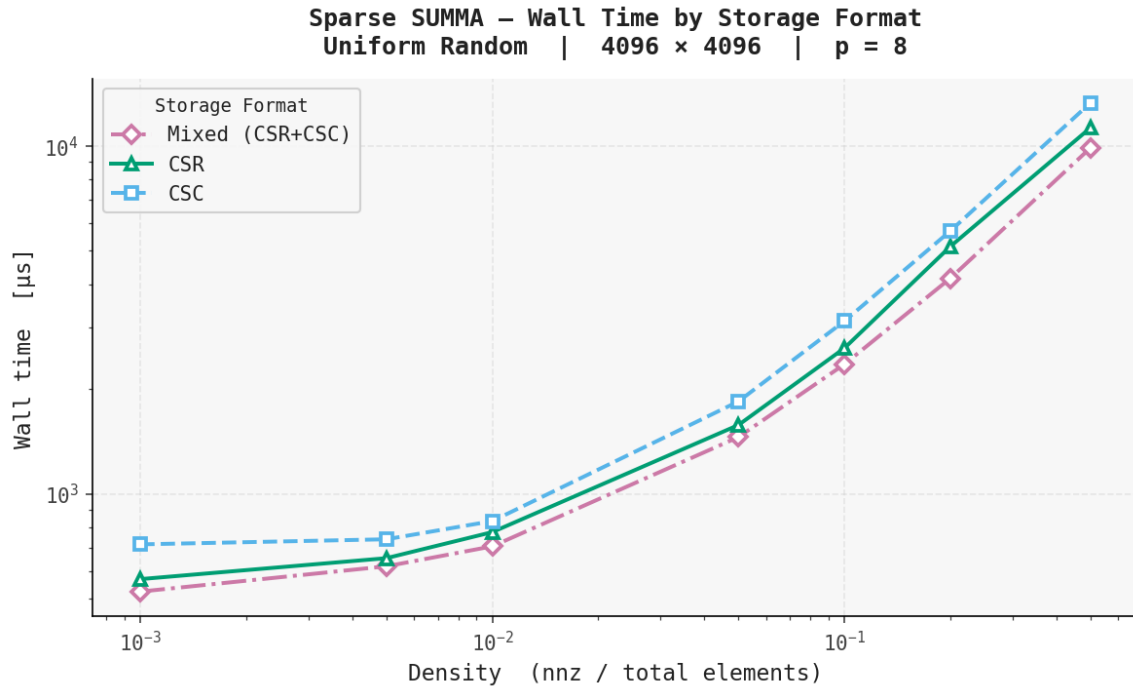


Figure 6.20

6.4.5 Communication vs Computation Bottlenecks

As noted in Chapter 2, an important topic to consider is whether or not our sparse SUMMA algorithm is actually being limited by the communication speeds or if we are still running into the dense problem of computation. Via our earlier cost analysis in Chapter 3, the expected result should be that our algorithm is almost completely bottlenecked by communication overhead. Figure 6.21 breaks down the sparse SUMMA wall time into three components: data communication, metadata overhead, and local SpGEMM computation, across a range of densities.

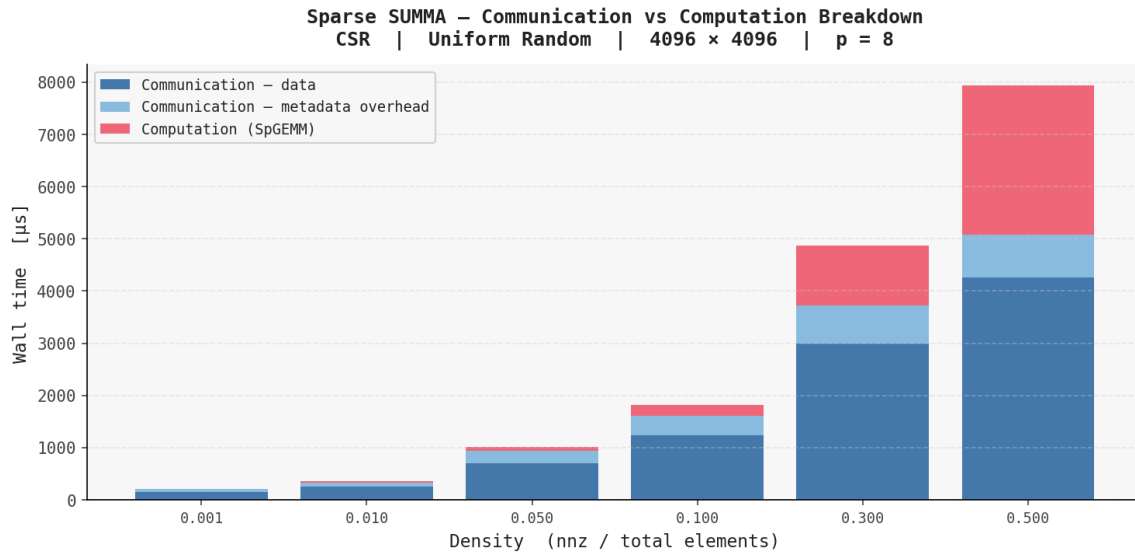


Figure 6.21

The results confirm that the communication dominates all of the density levels we tested. At around 10% density, data communication already accounts for a majority of the total runtime with computation contributing a small fraction of it. As the density increases towards 50% it becomes clear that computation is starting to become a bottleneck because the panels are becoming dense enough for SpGEMM to perform lots of work. The metadata overhead does remain small overall compared to the

data communication, but at these sizes and communication costs is not a significant bottleneck.

6.4.6 Load Imbalance Observations

Figure 6.22 shows the distribution of non zero entries per rank across the 8 processes for three sparsity patterns. The dashed line marks the mean non zeroes per rank, which is held constant across all of the patterns.

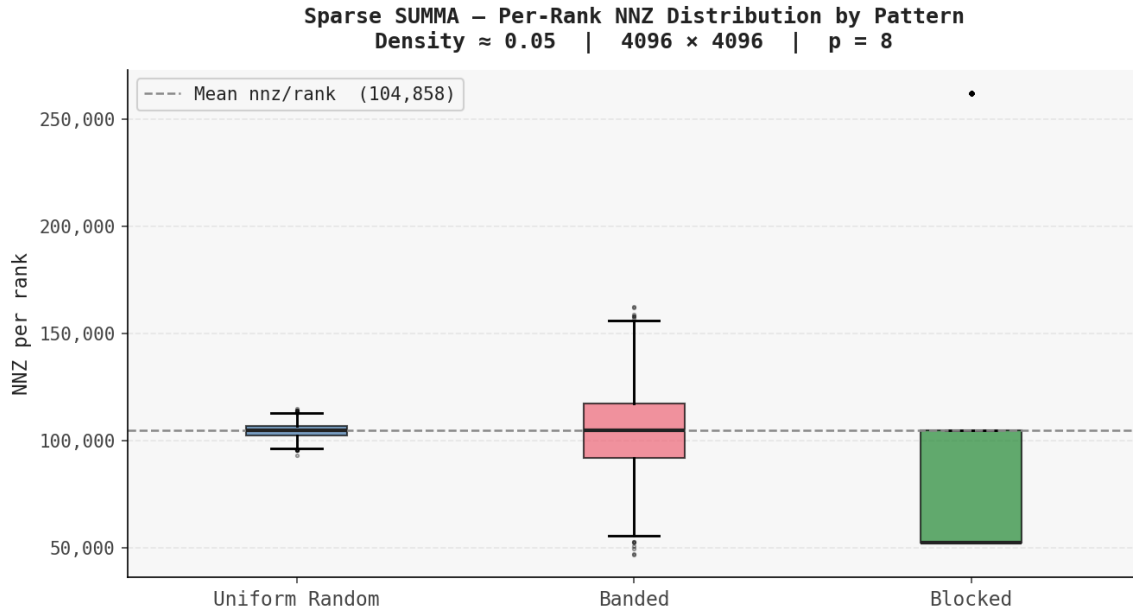


Figure 6.22

As expected, the uniform sparsity pattern produces a tight distribution with minimal spread around the mean. This confirms that the balanced baseline as described in Chapter 3 is correct, where nearly every rank holds approximately the same number of non zeroes.

The banded sparsity pattern shows a substantially higher variance. Because the band concentrates non zeroes into a contiguous row, the processes covering that region

receive a disproportionate share of the work. This corresponds directly into the per step communication imbalance referred to in Section 3.3.3, where the slowest process determines the wall time of each collective.

The blocked pattern shows the widest distribution and the heaviest outliers. Because non zero entries are concentrated in these dense rectangular blocks, a rank that happens to own rows intersecting one or more blocks receives a disproportionate share of the total non zeroes. This produces the distribution we see in the diagram, some ranks are heavily loaded and others are nearly idle.

Taken together, these distributions explain a problem that appears throughout the SUMMA results: a sparsity pattern can have a low average communication cost yet still performs poorly due to load imbalance. Conversely, a pattern with a higher average cost but uniform distribution can achieve more consistent performance when scaling.

6.4.7 Combined Effects

Figure 6.23 presents a grid of wall time heatmaps combining the effects of sparsity pattern, density, and process count for CSR storage on a 4096×4096 matrix. Each sub grid corresponds to a fixed process count where $p = 2, 4, 6, 8$, with rows representing the patterns and columns representing the density levels.

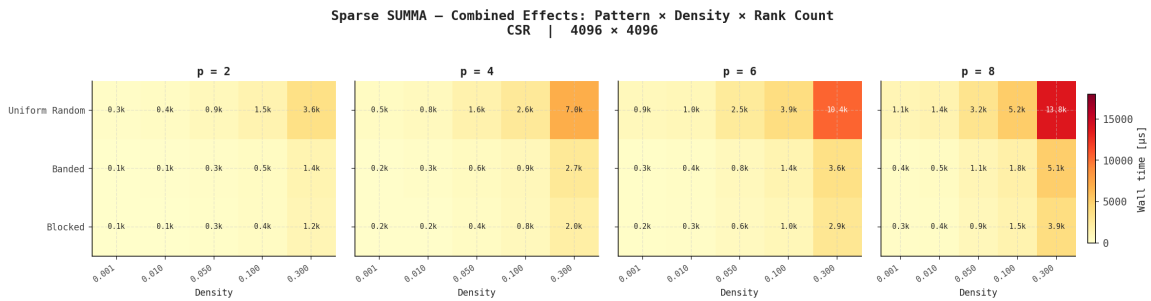


Figure 6.23

This figure demonstrates a few trends that occur from sparse SUMMA. First, the sparsity pattern dominates at low process counts and low densities. At $p = 2$, the gap between uniform random and structured patterns is quite large across all densities. Second, as the process count increases the gap between the patterns narrow at low densities because the per step startup latency α becomes a larger fraction of the total cost. Third, the hottest cells in the grid, or notably the higher wall times, consistently appear in the uniform patterns. The high density, high process count corners essentially represent the worst case combination of maximum communication volume per step and maximum latency.

6.5 Discussion

The experimental results we presented largely confirm the predictions we created earlier in this thesis. This section takes all of that information and creates rough guidance for selecting storage formats and communication strategies based on known properties like matrix size and density.

6.5.1 Storage Format Selection

Across both the collective communication experiments and the SUMMA results, the relative performance of COO, CSR, and CSC follow consistent patterns. COO is the slowest format for communication heavy workloads due to its higher metadata overhead, transmitting two full integer arrays of length nnz alongside the values. The performance gap between COO and the compressed formats widens as the density increases, which matches the cost models we developed earlier. At very low densities, the formats converse due to the total message size being small enough that the α term dominates over bandwidth.

CSR and CSC perform nearly identically for symmetric operations like broadcast and gather when the matrix is square and uniform. The main distinction here is only when the operation being performed has a directional preference. For example, in the SUMMA context we discussed the A panel extraction benefits from CSR and the B panel extraction benefits from CSC. The mixed CSR and CSC strategy provides the best efficiency, for SUMMA at low densities its advantage over single format (CSC or CSR only) starts wearing off around 5% to 10%.

The main outcome here is fairly clear - for general collective operations on sparse data CSR or CSC should be preferred over COO except for when construction simplicity is a main concern. For distributed sparse matrix multiplication specifically, a mixed strategy is definitely worth the additional complexity when operating in specific sparse patterns, but can be safely simplified to a single format at higher densities without meaningful loss.

6.5.2 Sparsity Pattern and Algorithm Selection

6.5.3 Communication as the Dominant Bottleneck

6.5.4 Scaling Behavior

6.5.5 Summary

Chapter 7

Distributed Sparse Loading and Storage

The experimental evaluation of distributed sparse algorithms requires that large sparse matrices be loaded from disk and distributed across the process grid efficiently. This loading step itself is a distributed operation with its own communication costs and load imbalance challenges. This chapter describes the problem of distributed sparse loading, the strategies implemented to address it, and some tradeoffs. In addition, it covers a possible solution that involves preprocessing data into a format that is better suited for distributed loading.

7.1 The Distributed Loading Problem

Sparse matrices used in scientific computing when stored physically (e.g. on a file system) are commonly stored in the Matrix Market format, which represents the non zero structure as a text file contain one triple (row, column, value) per line [20]. The format is widely supported but presents several challenges for distributed loading. The

file is inherently sequential: entries are listed in order, typically sorted by row, and the total file size for large matrices can reach dozens of gigabytes if not larger. Reading the entire file on a single process and then distributing the data is straightforward to implement, but creates a single node memory bottleneck and a communication bottleneck as the root process sends data to all other processes. In most cases on big datasets, this is not feasible as a single process cannot physically hold enough data in memory to be able to distribute it. Alternatively, the process can load in chunks of data at a time but that is limited by processor speed, available memory, and the communication speed at which the root process can send data to other processes.

7.1.1 Loading Strategies

For all of the mentioned strategies, there is a shared set of tooling that we can use for implementation. While the primary focus of OpenMPI is to handle the communication and organization of processes or nodes, another use case is its ability to handle the distributed loading of files. Listing 8 shows a heavily abstracted version of a row based distributed data loader, particularly highlighting the usage of the OpenMPI file operations.

```

1  if (rank == 0) {
2      MPI_File_open(MPI_COMM_SELF, filename, MPI_MODE_RDONLY, &file);
3
4      parse_header(file, &nrows, &ncols, &nnz);
5      // ... assign byte offsets for each process
6
7      MPI_File_close(&file);
8  }
9
10 // ... broadcast header and index to all processes
11 // ... each process computes local index math
12
13 // Open the file and grab our data
14 MPI_File_open(MPI_COMM_WORLD, filename, MPI_MODE_RDONLY, &file);
15 MPI_File_read_at_all(file, read_offset, local_buffer, read_length,
    ↪ MPI_CHAR, &status);
16 MPI_File_close(&file);
17
18 for each entry (row, col, val) parsed from local_buffer {
19     local_coo.rows.push(row);
20     local_coo.cols.push(col);
21     local_coo.values.push(val);
22 }

```

Listing 8: An example of distributed data loading using OpenMPI file operations.

7.1.2 Single Process Read and Broadcast/Scatter

The simplest strategy as we mentioned earlier is when a single process reads in an entire file at once, or chunk by chunk, and distributes that data via any of the available MPI operations such as broadcast or scatter. In this case, we do not need to use any of the OpenMPI file operations and instead rely on barebones C/C++ function calls.

7.1.2.1 Row Based Partitioning

As the name implies, the core idea behind this partitioning strategy is to distribute an even number of rows across the processes. The implementation of this depends on the complexity of the storage format. For example, a basic matrix market format file could be row partitioned by default, or could just have a nonsensical organization non zero values. This complicates things whenever a format is not in row format, as we then have to load all available non zeroes and map them by row. While it is somewhat possible to have each process distribute rows to other processes based on their ranks, this would theoretically destroy any possible runtime performance.

7.1.2.2 Nonzero Balanced Partitioning

Nonzero balanced partitioning, again as the name implies, takes the total number of stored non zeroes and distributes them evenly across the processes. The implementation here does not depend on the storage format, it just needs a series of non zeroes to be available.

7.1.2.3 Tradeoffs

7.1.3 Storage Format Conversions

While using the Matrix Market format is fine for most cases, we did discover that there are tradeoffs to how we can load data from them. Particularly, if the data is not in the format we need (say partitioned by row), then it can be costly to repartition it at runtime. Alternatively, there is another solution that can reduce the runtime performance. By converting the local file to another format beforehand, one that is partitioned and possibly has metadata calculated ahead of time, we can drastically reduce the runtime cost of loading data from disk.

7.1.3.1 HDF5

The Hierarchical Data Format (HDF) format is an open source file format designed to store and manage large and complex data sets [21]. For the purposes of this thesis, we will be focusing on Version 5 of this format.

The HDF format is structured similar to a file system, where its split into a series of groups and datasets which are self described in the files metadata. This theoretically allows it to store any type of structure we want it to, whether that be row based partitions of sparse data or even block based partitioning.

7.1.3.2 Zarr

The Zarr file format is an open source and maintained specification for efficient and scalable storage of large arrays of data [22]. Interestingly, it is optimized for cloud storage and specifically cloud workloads where network bandwidth and hardware IO performance is a concern.

The core structure of the Zarr format is that data is broken into smaller and smaller chunks that are broken into multiple files. This allows for extremely efficient retrieval of small subsets of some larger data set.

7.2 Summary

Chapter 8

Visualization and STEM Communication

Outside of the implementation of sparse collective communications and algorithms, another problem that often arises is the ability to visually communicate how these algorithms work. Because they often work over multiple steps, as is seen in the SUMMA algorithm, its not uncommon for a problem to require either multiple figures or an animated figure of some sort to understand it. This chapter focuses on the work we did throughout this thesis to create visualizations and animations related to sparse algorithms. Although there are not any clear cut experiments or results outside of the visualizations and code to create them, it does create an interesting research topic for future work.

8.1 Communicating Sparse Structure

Talk about the differences in communicating sparse structures versus dense structures...

8.2 Communicating Sparse Algorithms

Talk about the differences in communicating sparse structures versus dense structures...

8.3 Design Principles for Technical Figures

What all should we keep in mind when developing these figures?

8.4 Figures in this Thesis

Many of the diagrams in this thesis were created using a custom tool designed to visualize sparse data but also visualize multiple the effects and contribution of multiple ranks. In general, there are not many preexisting tools that are able to easily visualize multi rank and sparse data easily. The following sections all demonstrate how the tool we created works by presenting examples of the code used to generate a variety of figures.

8.4.1 Tool Implementation

Here there should be a section that covers how this tool was created just outside of individual codes that generate matrix visualization, collective communications, etc.

8.4.2 Matrix Structure Diagrams

This section will specifically speak about the diagrams used to demonstrate matrix sparsity. Show an example of the pseudocode used to generate it as well as a figure that the code would produce.

8.4.3 Collective Communication Diagrams

For generating collective communications, a generic tool was created that can take any number of states and any number of processes and visualize the way that data flows between each state. Listing 9 demonstrates a subset of the code required to generate Figure 2.7. Left out of this listing is code for trivial tasks like importing from other files, the implementation of small functions like *mk*, etc.

```

1 <script>
2     const states = [
3         mk([[0, x]]),
4         mk([[0, x], [3, x]]),
5         mk([
6             [0, x], [1, x],
7             [2, x], [3, x],
8         ]),
9     ];
10    const stepLabels = ["Step 1", "Step 2"];
11    const messages = [
12        [{ from: 0, to: 3, label: "x" }],
13        [
14            { from: 0, to: 1, label: "x" },
15            { from: 3, to: 2, label: "x" },
16        ],
17    ];
18 </script>
19
20 <CollectiveDiagram
21     title="MST Broadcast"
22     subtitle="root = 0"
23     p={4}
24     {states}
25     {stepLabels}
26     {messages}
27 />

```

Listing 9: An example of the code for a MST Broadcast visualization.

This section will specifically speak about the collectives shown in Chapter 2. Show an example of the psuedocode used to generate it as well as a figure that the code would produce.

8.4.4 Performance and Scaling Charts

This one will be a bit different, and may even remove it, as this is just standard stuff (matplotlib, etc) for most common papers.

8.4.5 Summary

As previously mentioned, outside of the figures used to create this thesis and the individuals reading this thesis, there were no experiments or studies ran to determine the effectiveness of these figures.

Chapter 9

Discussion and Future Work

9.1 Summary of Contributions

This thesis has made four primary contributions to the understanding of sparse collective communication and distributed sparse matrix multiplication. First, it has provided a systematic analysis of how four distinct sparsity patterns: uniform random, banded, block, and Kronecker product affect the communication cost, load balance, and scaling behavior of standard collective communication operations adapted for sparse data. Prior work on sparse distributed computing has frequently focused on a single application or a single class of sparsity patterns, and this thesis was created to span multiple patterns under a shared cost model.

Secondly, it has extended the SUMMA algorithm to the sparse case and analyzed how the choice of storage format interacts with the two phase broadcast and multiply structure of the algorithm.

Third, it has characterized the distributed sparse loading problem and implemented and compared multiple partitioning strategies.

Fourth, it has demonstrated the need for additional research and work on visual-

ization technologies for sparse workloads.

9.2 Pattern-Algorithm Summary

The results of this thesis offer several guidelines for selecting communication strategies and storage formats based on known properties of input matrices.

For uniform random sparsity, we determined that

For banded sparsity, we determined that

For block sparsity, we determined that

For Kronecker product matrices, we determined that

9.3 Limitations

Several limitations of this thesis should be noted.

Xth, the SpGEMM kernel used in the sparse SUMMA implementation is straightforward and does not introduce any optimizations found in production libraries and frameworks such as CombBLAS. Computation time therefore represents an upper bound as to what is achievable for local matrix multiplication, and the communication to computation ratios may shift with a more optimized kernel, but is unlikely to change the overall results that we discussed.

9.4 Future Work

9.4.1 Extending to Other Sparse Formats

The analysis in this thesis focuses on the COO, CSR, and CSC formats, which are among the most commonly used formats in high performance sparse computing. How-

ever, a number of specialized formats offer performance advantages for specific sparsity patterns. For example, the ELLPACK format stores a fixed number of nonzeros per row in a dense array, which would be efficient for matrices with uniform row density but wastes space on skewed distributions [23]. We also discussed optimized Block Compressed Sparse Row earlier in Chapter 2.

9.4.2 Topology

An interesting extension to this thesis would be understanding network topology and how it effects the communication costs.

9.4.3 Adaptive Algorithm Selection

Bibliography

- [1] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003. DOI: [10 . 1137 / 1 . 9780898718003](https://doi.org/10.1137/1.9780898718003).
- [2] SciPy Developers, *SciPy sparse matrix documentation*, 2023. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/sparse.html>.
- [3] G. Huang, G. Dai, Y. Wang, and H. Yang, “GE-SpMM: General-purpose sparse matrix–matrix multiplication on GPUs for graph neural networks,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC’20)*, 2020. DOI: [10 . 1109 / SC41405 . 2020 . 00076](https://doi.org/10.1109/SC41405.2020.00076).
- [4] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank citation ranking: Bringing order to the web,” Stanford InfoLab, Tech. Rep. 1999-66, 1999. [Online]. Available: <http://ilpubs.stanford.edu:8090/422/>.
- [5] G. Strang, *Computational Science and Engineering*. Wellesley-Cambridge Press, 2007, ISBN: 9780961408817.
- [6] E.-J. Im, K. Yelick, and R. Vuduc, “Sparsity: Optimization framework for sparse matrix kernels,” 1, vol. 18, 2004, pp. 135–158. DOI: [10 . 1177 / 1094342004041296](https://doi.org/10.1177/1094342004041296).

- [7] R. W. Vuduc and H.-J. Moon, “Fast sparse matrix-vector multiplication by exploiting variable block structure,” in *Proceedings of the International Conference on High Performance Computing and Communications (HPCC)*, ser. Lecture Notes in Computer Science, vol. 3726, Springer, 2005, pp. 807–816. DOI: [10.1007/11557654_91](https://doi.org/10.1007/11557654_91).
- [8] J. Leskovec, D. Chakrabarti, J. Kleinberg, and C. Faloutsos, “Realistic, mathematically tractable graph generation and evolution, using Kronecker multiplication,” in *European Conference on Principles of Data Mining and Knowledge Discovery (PKDD)*, Springer, 2005, pp. 133–145. DOI: [10.1007/11564096_9](https://doi.org/10.1007/11564096_9).
- [9] J. Leskovec, D. Chakrabarti, J. Kleinberg, C. Faloutsos, and Z. Ghahramani, “Kronecker graphs: An approach to modeling networks,” *Journal of Machine Learning Research*, vol. 11, pp. 985–1042, 2010.
- [10] Graph500 Steering Committee, *Graph500 benchmark specification*, 2010. [Online]. Available: <https://graph500.org>.
- [11] Message Passing Interface Forum, “MPI: A message-passing interface standard, version 4.1,” University of Tennessee, Tech. Rep., 2023. [Online]. Available: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf>.
- [12] E. Gabriel et al., “Open MPI: Goals, concept, and design of a next generation MPI implementation,” in *Proceedings, 11th European PVM/MPI Users’ Group Meeting*, ser. Lecture Notes in Computer Science, vol. 3241, Budapest, Hungary: Springer, 2004, pp. 97–104.
- [13] E. Chan, M. Heimlich, A. Purkayastha, and R. van de Geijn, “Collective communication: Theory, practice, and experience,” *Concurrency and Computation: Practice and Experience*, vol. 19, no. 13, pp. 1749–1783, 2007. DOI: [10.1002/cpe.1206](https://doi.org/10.1002/cpe.1206).

- [14] R. A. van de Geijn and J. Watts, “SUMMA: Scalable universal matrix multiplication algorithm,” *Concurrency: Practice and Experience*, vol. 9, no. 4, pp. 255–274, 1997. DOI: [10.1002/\(SICI\)1096-9128\(199704\)9:4<255::AID-CPE250>3.0.CO;2-2](https://doi.org/10.1002/(SICI)1096-9128(199704)9:4<255::AID-CPE250>3.0.CO;2-2).
- [15] A. Buluç and J. R. Gilbert, “Parallel sparse matrix–matrix multiplication and indexing: Implementation and experiments,” 4, vol. 34, 2012, pp. C170–C191. DOI: [10.1137/110848244](https://doi.org/10.1137/110848244).
- [16] OU Supercomputing Center for Education and Research (OSCER), *OU supercomputing center for education and research*, <https://www.ou.edu/oscer>, Norman, OK.
- [17] A. Buluç and J. R. Gilbert, “The combinatorial BLAS: Design, implementation, and applications,” *International Journal of High Performance Computing Applications*, vol. 25, no. 4, pp. 496–509, 2011. DOI: [10.1177/1094342011403516](https://doi.org/10.1177/1094342011403516).
- [18] A. Azad, O. Selvitopi, M. T. Hussain, J. R. Gilbert, and A. Buluç, “Combinatorial BLAS 2.0: Scaling combinatorial algorithms on distributed-memory systems,” *IEEE Transactions on Parallel and Distributed Systems*, 2021. DOI: [10.1109/TPDS.2021.3094895](https://doi.org/10.1109/TPDS.2021.3094895).
- [19] B. Mohr, “PMPI tools,” in *Encyclopedia of Parallel Computing*, D. Padua, Ed., Boston, MA: Springer, 2011. DOI: [10.1007/978-0-387-09766-4_57](https://doi.org/10.1007/978-0-387-09766-4_57).
- [20] R. F. Boisvert, R. Pozo, and K. Remington, *The Matrix Market exchange formats: Initial design*, 1996. [Online]. Available: <https://math.nist.gov/MatrixMarket/formats.html>.
- [21] The HDF Group, *Hierarchical Data Format, version 5*, <https://www.hdfgroup.org/HDF5/>, 1997–2024.

- [22] A. Miles et al., *zarr-developers/zarr-python*, <https://doi.org/10.5281/zenodo.3773449>, 2023. DOI: [10.5281/zenodo.3773449](https://doi.org/10.5281/zenodo.3773449).
- [23] N. Bell and M. Garland, “Implementing sparse matrix-vector multiplication on throughput-oriented processors,” in *Proceedings of the ACM/IEEE Conference on Supercomputing (SC’09)*, 2009. DOI: [10.1145/1654059.1654078](https://doi.org/10.1145/1654059.1654078).